testgen: Word test description to tpf test file

A hybrid pipeline for a small offline LLM (for example Gemma 9B 4-bit). Code does everything exact: reading the document, decoding tables, copying values, building masks, writing the file. The model only makes narrow judgement calls (what a section is, what a column means) and always **chooses from lists that code computed**; it never types a variable name or a value that ends up in the output. Built with LangGraph orchestration and LangChain model access (Ollama or LM Studio).

1. The flow



Blue = code only, orange = LLM plus code, green = rules first, the LLM is asked only for ambiguous cases. Every stage writes a checkpoint, so you can fix a decision by hand and continue with `--resume-from <stage>`.

2. What each stage does

Stage	Input -> output	Decision logic and safeguards
0 load	docx -> blocks (heading, paragraph, list, table)	Reads body in document order (also inside content controls). Tables are decoded from the XML grid (gridSpan/vMerge), not from python-docx cells. If no heading styles exist, short bold/numbered paragraphs are promoted and an info issue says so. Corrupt, empty, .doc or wrong files stop the run with a clear message.
1 sections	blocks -> sections with labels	Keyword rule gives a hint; LLM labels every section with an enum (index, document_info, requirements, initialization, test_inputs, expected_outputs, constraints, other). Batches with overlap for long documents; previews shrink to fit the prompt budget. Rule/LLM disagreement is reported. LLM failure: rule label, else 'other' + review item.
2 roles	tables -> input / expected / mixed / ignore	Section label decides when unambiguous. Several data tables in one section, or a heading that contradicts the label, goes to the LLM; fallback is keywords. At least one input or expected table is required, otherwise an error issue.
3 recipes	table layout -> column meanings	Per field (lane groups count as ONE field): config map, then ignore patterns (Remarks...), then exact/fuzzy match to symbols.csv, then LLM picks one of at most 5 code-computed candidates or NONE. Unresolved columns are kept and flagged. Cached by layout in recipe_dir; set approved=true after checking.
4 assemble	recipes + rows -> test cases	Joins input and expected rows by id column if ids are unique, else by position. Count/id mismatches are reported and no row is dropped. Same-layout tables are concatenated. Environment (func, computers) only from mapped columns or case_environment_default; otherwise ONE review issue.
5 notes	init + constraint paragraphs -> hints	Two passes (rewrite, then JSON). Every item needs an evidence quote that must occur in the source text; otherwise it is dropped and reported.
6 ground	cases + notes -> spec.json	Attaches symbols; variables missing from symbols.csv become unresolved; remaining free-text names are resolved by rules, then LLM choice among candidates. Writes the hand-editable spec.
7 validate	spec -> issues	Unresolved items, lane count vs num_computers, conflicting values for overlapping computers, empty cases, sections/tables that nothing used.
8 render	spec -> .tpf	Pure templates from config. Channel lanes become '# TODO' lines (syntax unknown) or masks with input_lane_render='mask'. Unresolved items are written as ??UNRESOLVED?? lines, never skipped.

3. Rendering rules

Case	Output
scalar input	`sym[111] <-> value` (default input mask)
scalar expected	`sym[1111] <-> value/` (default check mask)
4 lanes [1,0,1,0]	`sym[1010] <-> 1/` and `sym[0101] <-> 0/` - computers with equal values share a line
lane value '-' or empty	don't care: the computer is left out of every mask
leading zeros	kept (`0110` stays `0110`); all values are strings
manual verify	`sym = /`

4. What happens when something goes wrong

Situation	Behaviour
Model offline, wrong port, library missing	Every call fails as LLMError; each stage falls back to rules; run finishes; review.md lists what was not decided by the model. `--backend none` does this on purpose.
Model returns prose, bad JSON, wrong enum, missing key	Fences/chatter stripped; JSON validated against the schema; on failure retried with the exact error appended to the prompt (default 2 retries); then fallback.
Output cut off at token limit	Detected from done_reason / finish_reason = length; treated as invalid and retried.
Model hangs	Per-call wall-clock timeout (default 120 s) in a daemon thread; counts as a failed attempt; transport errors back off 1 s, 2 s...
Server rejects the schema / old library	Transport drops constrained decoding (format schema, then plain JSON mode, then none) and keeps going; answers are still schema-validated in code.
Model invents a variable or a quote	Variables must be one of the candidate names (enum + re-check). Notes need an evidence quote found in the source, else dropped and reported.
A stage throws an exception	Caught per node; traceback to work/errors.log; rule-based fallback result; error issue; run continues. --strict (or the load stage) makes it fatal.
Weird table (orphan vMerge, gridBefore, 5000 rows, huge span, no rows)	Decoder clamps and warns; undecodable tables keep a text fallback and a review issue; nothing crashes.
Corrupt/old/empty docx, bad config, bad symbols	FatalError with a clear message, exit code 2, review.md still written.
Cache or log file unwritable/corrupt	Ignored (best effort); recipes and LLM answers are recomputed.
LangGraph itself fails or is missing	engine=auto uses the built-in sequential runner (same node functions and reducers); a failing LangGraph run is re-run sequentially and noted.
Same finding on every row	Aggregated per column in review.md (for example '4 occurrences, first shown').

5. Running it

```
pip install -r requirements.txt
python -m testgen check  --symbols symbols.csv --backend ollama --model gemma2:9b   # config + connection
python -m testgen tables spec.docx                          # decoder preview, no LLM, no symbols needed
python -m testgen run    spec.docx --symbols symbols.csv --config config.json --out out/
python -m testgen run    spec.docx --symbols symbols.csv --backend none            # rules only
python -m testgen run    spec.docx --symbols symbols.csv --resume-from recipes     # after hand edits
python -m testgen render out/spec.spec.json                 # re-render a hand-fixed spec
python -m testgen eval   out/spec.tpf gold.tpf              # precision/recall against a gold file
# LM Studio: --backend lmstudio --model <name> --base-url http://localhost:1234/v1
# exit code: 0 clean | 1 items to review | 2 fatal
```

Output folder: `name.tpf, name.spec.json, name.review.md, name.tables.md, name.run_report.json, work/` (checkpoints 00..08, llm_log.jsonl with every prompt and answer, llm_cache/, errors.log).

symbols.csv needs the columns `variable,symbol`; optional `description` and `aliases` (separated by |). Aliases are the cheapest way to fix a column that does not match.

6. Decisions you may want to change (all in config.json)

Setting	Default	Meaning
lane_meaning	check: computer, set_input: channel	How sub-columns 1..n are read. Only 'computer' lanes become masks.
input_lane_render	todo	Channel lanes: '# TODO' line + review item. 'mask' treats them like computers.
default_masks	input 111, check 1111	Mask for scalar values.
case_environment_default	null	func + target_computer for every case when no column provides them.
table_column_map	{}	Force a column: 'Header': 'variable' or {variable, kind, lane_meaning}; kinds include env_func, env_computers, env_verify.
table_options	auto	header_rows, transpose, id_column, per_table overrides when the header detection is wrong.

Setting	Default	Meaning
emit_global_init	false	Write the initialization hints into the tpf (otherwise only review.md).
formats	see config.py	Every output line is a template; syntax changes need no code.

7. Verified and not verified

Verified here (129 automated tests, all passing): decoder on merged/lane/counter-row tables and malformed XML; masks; grounding; schema validator; LLM client (retry, fences, truncation, timeout, cache, disabled); the whole pipeline on synthetic documents (with and without LLM, without headings, reordered, extra tables, row mismatch, prose only, corrupt inputs); injected failures for every LLM task and mode; stage crashes and strict mode; resume; recipe cache; CLI exit codes; the LangGraph wiring against a stand-in that enforces the same rules.

Not verified here: this sandbox has no network, so the real `langgraph`, `langchain-ollama`, `langchain-openai` and a real model were never run. The graph code uses only StateGraph, START, END, add_node, add_edge, add_conditional_edges and compile; if your langgraph version misbehaves, `--engine sequential` gives identical results. Run `python -m testgen check` first to test the model connection. Prompt quality for Gemma 9B is also untested; the first real run will show which columns need aliases or table_column_map entries. The table decoder is a rewrite (your earlier decoder file was not available in this chat); anything returning the same dict shape (see table_decoder.py docstring) can replace it.

Appendix: complete source code

Line numbers are for reference. Order follows the data flow. All code is plain Python 3.9+; only python-docx is needed for the core, LangGraph/LangChain are imported lazily.

`testgen/__init__.py` **(2 lines)**

```
"""testgen - Word test description -> tpf test file (hybrid LLM + code pipeline)."""
__version__ = "1.0.0"
```

`testgen/issues.py` **(73 lines)**

```
"""Issue records and the fatal-error type shared by every stage.

An *issue* is a plain dict so it can be stored in JSON checkpoints and merged
by LangGraph reducers:

    {"severity": "error" | "review" | "info",
     "stage": "load" | "sections" | ...,
     "location": "T2 row 3 / Mode",          # where it happened (free text)
     "message": "what is wrong",
     "fix": "what a human can do about it"}

Severity meaning
    error   something failed or is certainly wrong in the output
    review  output was produced but a human must check it
    info    harmless note (kept for traceability)

Exit codes of the CLI: 0 = clean, 1 = finished with error/review items,
2 = fatal (run could not finish).
"""
from __future__ import annotations

from typing import Any, Dict, Iterable, List

SEVERITIES = ("error", "review", "info")


class FatalError(Exception):
    """The run cannot continue (bad input file, bad config, missing checkpoint)."""


def make_issue(severity: str, stage: str, message: str,
               location: str = "", fix: str = "") -> Dict[str, str]:
    """Build one issue dict. Unknown severities are downgraded to 'review'."""
    if severity not in SEVERITIES:
        severity = "review"
    return {
        "severity": severity,
        "stage": str(stage),
        "location": str(location),
        "message": str(message),
        "fix": str(fix),
    }


def dedupe(issues: Iterable[Dict[str, Any]]) -> List[Dict[str, Any]]:
    """Drop exact duplicates (same severity, stage, location, message), keep order."""
    seen = set()
    out: List[Dict[str, Any]] = []
    for item in issues:
        key = (item.get("severity"), item.get("stage"),
               item.get("location"), item.get("message"))
        if key in seen:
            continue
        seen.add(key)
        out.append(item)
    return out


def counts(issues: Iterable[Dict[str, Any]]) -> Dict[str, int]:
    """Number of issues per severity."""
    result = {s: 0 for s in SEVERITIES}
    for item in issues:
        sev = item.get("severity", "review")
        result[sev] = result.get(sev, 0) + 1
    return result


def exit_code(issues: Iterable[Dict[str, Any]], fatal: bool) -> int:
    """0 clean, 1 finished with items to review, 2 fatal."""
    if fatal:
        return 2
    c = counts(issues)
```

```
    return 1 if (c["error"] or c["review"]) else 0
```

testgen/config.py **(181 lines)**

```
"""Configuration loading and validation.

`load_config(path)` merges a user JSON file over DEFAULT_CONFIG and validates
it. A broken config is a FatalError with a message that says what to fix.
Unknown top-level keys are reported as review issues (they are usually typos).
"""
from __future__ import annotations

import copy
import json
import os
import string
from typing import Any, Dict, List, Optional, Tuple

from .issues import FatalError, make_issue

SECTION_LABELS = ["index", "document_info", "requirements", "initialization",
                  "test_inputs", "expected_outputs", "constraints", "other"]

# Output syntax templates. Placeholders allowed per template are listed in
# FORMAT_FIELDS; anything else is rejected at load time so a typo in the config
# cannot crash rendering halfway through a run.
DEFAULT_FORMATS = {
    "section_init": "# init system and verify",
    "init_func": "func <-> {name}",
    "init_check": "{sym} <-> {value} /",
    "init_verify": "{sym} = /",
    "case_header": "# test case {n}",
    "step_header": "# step {n}",
    "section_env": "# setup test case environment",
    "env_func": "func <-> {name}",
    "env_computers": "pcomp <-> {mask}",
    "section_env_verify": "# test and verify test case environment",
    "env_check": "{sym} <-> {value} /",
    "env_verify": "{sym} = /",
    "section_inputs": "# setting inputs",
    "set_input": "{sym}[{mask}] <-> {value}",
    "section_checks": "# checking expected output",
    "check": "{sym}[{mask}] <-> {value}/",
    "verify": "{sym} = /",
    "todo": "# TODO {text}",
    "unresolved": "??UNRESOLVED: {text}??",
}
FORMAT_FIELDS = {
    "section_init": set(), "init_func": {"name"}, "init_check": {"sym", "value"},
    "init_verify": {"sym"}, "case_header": {"n", "id"}, "step_header": {"n"},
    "section_env": set(), "env_func": {"name"}, "env_computers": {"mask"},
    "section_env_verify": set(), "env_check": {"sym", "value"}, "env_verify": {"sym"},
    "section_inputs": set(), "set_input": {"sym", "mask", "value"},
    "section_checks": set(), "check": {"sym", "mask", "value"}, "verify": {"sym"},
    "todo": {"text"}, "unresolved": {"text"},
}

DEFAULT_KEYWORDS = {
    "index": ["index", "table of contents", "contents", "toc"],
    "document_info": ["author", "document information", "document info", "revision",
                      "history", "version", "scope", "introduction", "purpose",
                      "references", "glossary", "abbreviation", "approval"],
    "requirements": ["requirement", "specification", "traceab"],
    "initialization": ["initiali", "precondition", "pre condition", "prerequisite",
                       "setup", "set up", "boot"],
    "test_inputs": ["test input", "input", "stimul", "test case", "test step",
                    "test procedure", "test sequence", "test scenario"],
    "expected_outputs": ["expected", "output", "result", "verification", "acceptance",
                         "pass criteria"],
    "constraints": ["constraint", "assumption", "limitation", "note", "remark",
                    "restriction", "caution", "warning"],
}

DEFAULT_CONFIG: Dict[str, Any] = {
    "funcs": [],                    # known function names; empty = not enforced
    "num_computers": 4,             # lanes of an expected-output field = computers
    "default_masks": {"input": "111", "check": "1111"},
    "lane_meaning": {"check": "computer", "set_input": "channel"},
    "input_lane_render": "todo",       # "todo" or "mask" (treat channel lanes like computers)
    "dont_care_values": ["", "-", "--", "x", "X", "n/a", "N/A", "na", "NA", "\u2014", "\u2013"],
    "env_expectations": {},            # {"in_water": {"vehicle_mode": "2"}}
    "case_environment_default": None,  # {"func": ["in_water"], "target_computer": [1,1,1,1]}
    "table_options": {"header_rows": None, "transpose": False, "id_column": None,
                      "text_format": "plain", "per_table": {}},
    "table_column_map": {},            # header text -> variable, or {"variable","kind","lane_meaning"}
    "ignore_header_patterns": ["^remarks?$", "^comments?$", "^notes?$", "^description$",
                               "^requirement(s)?( id)?$", "^reference(s)?$", "^status$",
                               "^tester$", "^date$", "^pass\\s*/\\s*fail$"],
```

```
    "keyword_rules": DEFAULT_KEYWORDS,
    "sections": {"batch_size": 12, "overlap": 2, "preview_chars": 300,
                 "max_prompt_chars": 6000},
    "grounding": {"accept": 0.92, "margin": 0.05, "floor": 0.55, "top_n": 5},
    "llm": {"backend": "ollama", "model": "gemma2:9b", "base_url": None,
            "temperature": 0.0, "max_tokens": 1024, "timeout": 120.0, "retries": 2,
            "backoff": 1.0, "num_ctx": None, "fold_system": True, "cache": True,
            "log_full": True},
    "formats": DEFAULT_FORMATS,
    "emit_global_init": False,
    "recipe_dir": "recipes",
    "engine": "auto",                 # auto | langgraph | sequential
    "strict": False,
}

_TYPES = {
    "funcs": list, "num_computers": int, "default_masks": dict, "lane_meaning": dict,
    "input_lane_render": str, "dont_care_values": list, "env_expectations": dict,
    "table_options": dict, "table_column_map": dict, "ignore_header_patterns": list,
    "keyword_rules": dict, "sections": dict, "grounding": dict, "llm": dict,
    "formats": dict, "emit_global_init": bool, "recipe_dir": str, "engine": str,
    "strict": bool,
}


def _merge(base: Any, override: Any) -> Any:
    """Recursive dict merge; lists and scalars from `override` replace `base`."""
    if isinstance(base, dict) and isinstance(override, dict):
        out = copy.deepcopy(base)
        for key, value in override.items():
            out[key] = _merge(base[key], value) if key in base else copy.deepcopy(value)
        return out
    return copy.deepcopy(override)


def validate_config(cfg: Dict[str, Any]) -> List[Dict[str, str]]:
    """Check types and templates. Raise FatalError for anything unusable."""
    issues: List[Dict[str, str]] = []
    for key in cfg:
        if key not in DEFAULT_CONFIG:
            issues.append(make_issue("review", "config", "unknown config key %r (typo?)" % key,
                                     fix="check the spelling against config.example.json"))
    for key, typ in _TYPES.items():
        if key in cfg and not isinstance(cfg[key], typ) or (typ is int and isinstance(cfg.get(key), bool)):
            raise FatalError("config: %r must be of type %s" % (key, typ.__name__))
    if cfg["num_computers"] < 1 or cfg["num_computers"] > 16:
        raise FatalError("config: num_computers must be between 1 and 16")
    if cfg["input_lane_render"] not in ("todo", "mask"):
        raise FatalError("config: input_lane_render must be 'todo' or 'mask'")
    if cfg["engine"] not in ("auto", "langgraph", "sequential"):
        raise FatalError("config: engine must be auto, langgraph or sequential")
    for name, mask in cfg["default_masks"].items():
        if name not in ("input", "check") or not isinstance(mask, str) or set(mask) - set("01") or not mask:
            raise FatalError("config: default_masks.%s must be a string of 0/1 digits" % name)
    for name, value in cfg["lane_meaning"].items():
        if value not in ("computer", "channel", "bit", "none"):
            raise FatalError("config: lane_meaning.%s must be computer, channel, bit or none" % name)
    fmt = cfg["formats"]
    for name, template in fmt.items():
        if name not in FORMAT_FIELDS:
            issues.append(make_issue("review", "config", "unknown format key %r" % name))
            continue
        if not isinstance(template, str):
            raise FatalError("config: formats.%s must be a string" % name)
        try:
            used = {f for _, f, _, _ in string.Formatter().parse(template) if f}
        except ValueError as exc:
            raise FatalError("config: formats.%s is not a valid template: %s" % (name, exc))
        bad = used - FORMAT_FIELDS[name]
        if bad:
            raise FatalError("config: formats.%s uses unknown placeholder(s) %s; allowed: %s" % (
                name, sorted(bad), sorted(FORMAT_FIELDS[name])))
    for name in FORMAT_FIELDS:
        if name not in fmt:
            fmt[name] = DEFAULT_FORMATS[name]
    return issues


def load_config(path: Optional[str] = None,
                overrides: Optional[Dict[str, Any]] = None) -> Tuple[Dict[str, Any], List[Dict[str, str]]]:
    """Load defaults, merge the user file and CLI overrides, validate."""
    cfg = copy.deepcopy(DEFAULT_CONFIG)
    if path:
        if not os.path.isfile(path):
            raise FatalError("config file not found: %s" % path)
        try:
            with open(path, "r", encoding="utf-8-sig") as handle:
                user = json.load(handle)
        except ValueError as exc:
```

```
            raise FatalError("config file %s is not valid JSON: %s" % (path, exc))
        if not isinstance(user, dict):
            raise FatalError("config file %s must contain a JSON object" % path)
        cfg = _merge(cfg, user)
    if overrides:
        cfg = _merge(cfg, overrides)
    issues = validate_config(cfg)
    return cfg, issues
```

`testgen/schema.py` **(116 lines)**

```
"""JSON schemas for every LLM answer, plus a tiny validator.

Why a home-made validator: the schemas below are also what we hand to the
backend for constrained decoding (Ollama `format`, LM Studio `response_format`).
The backend only guarantees the *shape* when it supports grammars; this
validator re-checks the shape in code (so an unconstrained backend or an old
library version cannot silently let bad JSON through) and has no dependency.

Supported keywords: type, enum, required, properties, additionalProperties,
items, minItems, maxItems.
"""
from __future__ import annotations

from typing import Any, Dict, List, Sequence


def validate(instance: Any, schema: Dict[str, Any], path: str = "$") -> List[str]:
    """Return a list of human-readable problems (empty list = valid)."""
    errors: List[str] = []
    allowed = schema.get("enum")
    if allowed is not None:
        try:
            ok = instance in allowed
        except TypeError:
            ok = False
        if not ok:
            shown = ", ".join(repr(a) for a in list(allowed)[:12])
            more = "" if len(allowed) <= 12 else ", ..."
            errors.append("%s: %r is not one of [%s%s]" % (path, instance, shown, more))
            return errors
    kind = schema.get("type")
    if kind == "object":
        if not isinstance(instance, dict):
            return ["%s: expected an object, got %s" % (path, type(instance).__name__)]
        for key in schema.get("required", []):
            if key not in instance:
                errors.append("%s: missing required key %r" % (path, key))
        props = schema.get("properties", {})
        for key, value in instance.items():
            if key in props:
                errors.extend(validate(value, props[key], "%s.%s" % (path, key)))
            elif schema.get("additionalProperties") is False:
                errors.append("%s: unexpected key %r" % (path, key))
    elif kind == "array":
        if not isinstance(instance, list):
            return ["%s: expected an array, got %s" % (path, type(instance).__name__)]
        if "minItems" in schema and len(instance) < schema["minItems"]:
            errors.append("%s: needs at least %d items, got %d" % (path, schema["minItems"], len(instance)))
        if "maxItems" in schema and len(instance) > schema["maxItems"]:
            errors.append("%s: allows at most %d items, got %d" % (path, schema["maxItems"], len(instance)))
        item_schema = schema.get("items")
        if item_schema:
            for idx, item in enumerate(instance):
                errors.extend(validate(item, item_schema, "%s[%d]" % (path, idx)))
    elif kind == "string":
        if not isinstance(instance, str):
            errors.append("%s: expected a string, got %s" % (path, type(instance).__name__))
    elif kind == "integer":
        if isinstance(instance, bool) or not isinstance(instance, int):
            errors.append("%s: expected an integer" % path)
    elif kind == "number":
        if isinstance(instance, bool) or not isinstance(instance, (int, float)):
            errors.append("%s: expected a number" % path)
    elif kind == "boolean":
        if not isinstance(instance, bool):
            errors.append("%s: expected true or false" % path)
    return errors


def _obj(props: Dict[str, Any], required: Sequence[str] = ()) -> Dict[str, Any]:
    return {"type": "object", "properties": props,
            "required": list(required) if required else list(props.keys()),
            "additionalProperties": False}


def _enum(values: Sequence[str]) -> Dict[str, Any]:
    return {"type": "string", "enum": list(values)}
```

```

# ---------------------------------------------------------------------------
# One builder per LLM task. Every key is required, so the model cannot skip one.
# "reason" comes first on purpose: the model writes a short justification before
# it commits to the answer, which helps small models.
# ---------------------------------------------------------------------------
def section_labels_schema(ids: Sequence[str], labels: Sequence[str]) -> Dict[str, Any]:
    item = _obj({"id": _enum(ids), "label": _enum(labels)})
    return _obj({"labels": {"type": "array", "items": item,
                            "minItems": len(ids), "maxItems": len(ids)}})


def table_role_schema(roles: Sequence[str]) -> Dict[str, Any]:
    return _obj({"reason": {"type": "string"}, "role": _enum(roles)})


def field_map_schema(candidates: Sequence[str], kinds: Sequence[str]) -> Dict[str, Any]:
    return _obj({"reason": {"type": "string"},
                 "variable": _enum(list(candidates) + ["NONE"]),
                 "kind": _enum(kinds)})


def choose_variable_schema(candidates: Sequence[str]) -> Dict[str, Any]:
    return _obj({"variable": _enum(list(candidates) + ["NONE"])})


def notes_schema(funcs: Sequence[str] = ()) -> Dict[str, Any]:
    func_item: Dict[str, Any] = _enum(funcs) if funcs else {"type": "string"}
    test_var = _obj({"evidence": {"type": "string"}, "variable": {"type": "string"},
                     "expected_value": {"type": "string"}})
    verify_var = _obj({"evidence": {"type": "string"}, "variable": {"type": "string"}})
    constraint = _obj({"evidence": {"type": "string"}, "text": {"type": "string"}})
    return _obj({
        "funcs": {"type": "array", "items": func_item},
        "test_variables": {"type": "array", "items": test_var},
        "verify_variables": {"type": "array", "items": verify_var},
        "constraints": {"type": "array", "items": constraint},
    })
```

testgen/llm_client.py **(454 lines)**

```
"""Robust LLM access.

Layers
    Transport      one call: messages (+ optional JSON schema) -> Reply.
                   LangChainTransport talks to Ollama or LM Studio through
                   LangChain; ScriptedTransport (testing.py) is a fake.
    LLMClient      everything that makes a small local model dependable:
                   timeout, retry with backoff, truncation detection, code-fence
                   stripping, JSON extraction, schema validation, semantic
                   checks with error feedback, an on-disk cache and a call log.

Every failure ends as an LLMError subclass. Stages catch LLMError and fall back
to rules, so a bad model answer never crashes a run.
"""
from __future__ import annotations

import hashlib
import json
import os
import re
import threading
import time
from typing import Any, Callable, Dict, List, Optional, Sequence, Tuple

from .schema import validate as validate_schema

PROMPT_VERSION = "1"   # bump when prompts change so the cache is not reused


class LLMError(Exception):
    """Base class for every model-related failure."""


class LLMDisabled(LLMError):
    """Rules-only mode: no model is configured."""


class LLMTimeout(LLMError):
    """The model did not answer in time."""


class LLMTransportError(LLMError):
    """Connection problem, missing library, server error."""


class LLMOutputError(LLMError):
```

```
    """The model answered, but not with valid, acceptable JSON after all retries."""


class Reply:
    """One model answer. `truncated` is True when generation hit max_tokens."""
    __slots__ = ("text", "truncated")

    def __init__(self, text: str, truncated: bool = False) -> None:
        self.text = text
        self.truncated = truncated


Messages = List[Tuple[str, str]]          # [("system", text), ("human", text)]
Transport = Callable[[Messages, Optional[Dict[str, Any]]], Reply]


# ---------------------------------------------------------------------------
# helpers that clean up what small models return
# ---------------------------------------------------------------------------
_FENCE_RE = re.compile(r"^```[a-zA-Z0-9_-]*\s*\n?(.*?)\n?```\s*$", re.S)


def strip_fences(text: str) -> str:
    """Remove a surrounding ```json ... ``` fence if present."""
    stripped = text.strip()
    match = _FENCE_RE.match(stripped)
    return match.group(1).strip() if match else stripped


def extract_json(text: str) -> Any:
    """Parse JSON from model output, tolerating fences and chatter around it."""
    if len(text) > 400000:
        raise ValueError("model output is unreasonably long")
    cleaned = strip_fences(text)
    try:
        return json.loads(cleaned)
    except ValueError:
        pass
    decoder = json.JSONDecoder()
    for match in re.finditer(r"[\{\[]", cleaned):
        try:
            obj, _ = decoder.raw_decode(cleaned[match.start():])
            return obj
        except ValueError:
            continue
    raise ValueError("no JSON found in the model output")


def call_with_timeout(fn: Callable[[], Any], timeout: Optional[float]) -> Any:
    """Run fn in a daemon thread and give up after `timeout` seconds."""
    if not timeout or timeout <= 0:
        return fn()
    box: Dict[str, Any] = {}

    def runner() -> None:
        try:
            box["value"] = fn()
        except BaseException as exc:    # noqa: BLE001 - re-raised below
            box["error"] = exc

    thread = threading.Thread(target=runner, daemon=True)
    thread.start()
    thread.join(timeout)
    if thread.is_alive():
        raise LLMTimeout("no answer within %.0f s" % timeout)
    if "error" in box:
        raise box["error"]
    return box["value"]


def fold_system(messages: Messages) -> Messages:
    """Gemma has no system role: merge the system text into the first human turn."""
    system = [text for role, text in messages if role == "system"]
    rest = [(role, text) for role, text in messages if role != "system"]
    if not system:
        return rest
    if rest and rest[0][0] == "human":
        rest[0] = ("human", "\n".join(system) + "\n\n" + rest[0][1])
    else:
        rest.insert(0, ("human", "\n".join(system)))
    return rest


# ---------------------------------------------------------------------------
# LangChain transport (Ollama and LM Studio)
# ---------------------------------------------------------------------------
def _content_text(message: Any) -> str:
    content = getattr(message, "content", message)
    if isinstance(content, list):
```

```
        parts = []
        for part in content:
            if isinstance(part, dict):
                parts.append(str(part.get("text", "")))
            else:
                parts.append(str(part))
        return "".join(parts)
    return "" if content is None else str(content)


def _is_truncated(message: Any) -> bool:
    meta = getattr(message, "response_metadata", None) or {}
    reason = meta.get("done_reason") or meta.get("finish_reason")
    return reason == "length"


def _looks_like_schema_error(exc: BaseException) -> bool:
    text = str(exc).lower()
    return any(word in text for word in
               ("response_format", "json_schema", "grammar", "format", "schema", "unsupported"))


class LangChainTransport:
    """Chat model through LangChain.

    backend "ollama"   -> langchain_ollama.ChatOllama, schema passed as `format`
    backend "lmstudio" -> langchain_openai.ChatOpenAI pointed at LM Studio's
                          OpenAI-compatible server, schema passed as
                          response_format=json_schema
    If the server or library version rejects the schema the transport remembers
    that and continues without it (the client still validates every answer).
    Models are imported lazily so the rest of the package works without them.
    """

    def __init__(self, backend: str, model: str, base_url: Optional[str] = None,
                 temperature: float = 0.0, max_tokens: int = 1024, timeout: float = 120.0,
                 num_ctx: Optional[int] = None, fold: bool = True,
                 api_key: str = "lm-studio") -> None:
        self.backend = backend
        self.model = model
        self.base_url = base_url
        self.temperature = temperature
        self.max_tokens = max_tokens
        self.timeout = timeout
        self.num_ctx = num_ctx
        self.fold = fold
        self.api_key = api_key
        self._models: Dict[str, Any] = {}
        self._schema_ok = True

    # -- model construction ------------------------------------------------
    def _build_ollama(self, schema: Optional[Dict[str, Any]]) -> Any:
        try:
            from langchain_ollama import ChatOllama
        except ImportError as exc:
            raise LLMTransportError("langchain-ollama is not installed (pip install langchain-ollama)") from exc
        kwargs: Dict[str, Any] = {"model": self.model, "temperature": self.temperature,
                                  "num_predict": self.max_tokens}
        if self.base_url:
            kwargs["base_url"] = self.base_url
        if self.num_ctx:
            kwargs["num_ctx"] = self.num_ctx
        if schema is not None:
            for fmt in (schema, "json"):          # full schema first, plain JSON mode second
                try:
                    return ChatOllama(format=fmt, **kwargs)
                except Exception:                  # old library version: try the next option
                    continue
        return ChatOllama(**kwargs)

    def _build_openai(self, schema: Optional[Dict[str, Any]]) -> Any:
        try:
            from langchain_openai import ChatOpenAI
        except ImportError as exc:
            raise LLMTransportError("langchain-openai is not installed (pip install langchain-openai)") from exc
        model = ChatOpenAI(model=self.model, base_url=self.base_url or "http://localhost:1234/v1",
                           api_key=self.api_key, temperature=self.temperature,
                           max_tokens=self.max_tokens, timeout=self.timeout, max_retries=0)
        if schema is not None:
            model = model.bind(response_format={
                "type": "json_schema",
                "json_schema": {"name": "answer", "strict": True, "schema": schema}})
        return model

    def _get_model(self, schema: Optional[Dict[str, Any]]) -> Any:
        use_schema = schema if (schema is not None and self._schema_ok) else None
        key = json.dumps(use_schema, sort_keys=True) if use_schema is not None else ""
        if key not in self._models:
            if self.backend == "ollama":
```

```
                self._models[key] = self._build_ollama(use_schema)
            elif self.backend in ("lmstudio", "openai"):
                self._models[key] = self._build_openai(use_schema)
            else:
                raise LLMTransportError("unknown backend %r (use ollama or lmstudio)" % self.backend)
        return self._models[key]

    # -- the call ------------------------------------------------------------
    def __call__(self, messages: Messages, schema: Optional[Dict[str, Any]] = None) -> Reply:
        msgs = fold_system(messages) if self.fold else messages
        model = self._get_model(schema)
        try:
            answer = model.invoke(msgs)
        except LLMError:
            raise
        except Exception as exc:                    # noqa: BLE001
            if schema is not None and self._schema_ok and _looks_like_schema_error(exc):
                self._schema_ok = False             # server cannot do constrained output
                try:
                    answer = self._get_model(None).invoke(msgs)
                except Exception as exc2:           # noqa: BLE001
                    raise LLMTransportError("%s: %s" % (type(exc2).__name__, exc2)) from exc2
            else:
                raise LLMTransportError("%s: %s" % (type(exc).__name__, exc)) from exc
        return Reply(_content_text(answer), _is_truncated(answer))


# ---------------------------------------------------------------------------
# the client used by every stage
# ---------------------------------------------------------------------------
class LLMClient:
    def __init__(self, transport: Optional[Transport], model_id: str = "none",
                 retries: int = 2, backoff: float = 1.0, timeout: Optional[float] = 120.0,
                 cache_dir: Optional[str] = None, log_path: Optional[str] = None,
                 log_full: bool = True, sleep: Callable[[float], None] = time.sleep,
                 prompt_version: str = PROMPT_VERSION) -> None:
        self.transport = transport
        self.model_id = model_id
        self.retries = max(0, int(retries))
        self.backoff = backoff
        self.timeout = timeout
        self.cache_dir = cache_dir
        self.log_path = log_path
        self.log_full = log_full
        self.sleep = sleep
        self.prompt_version = prompt_version
        self.stats = {"calls": 0, "attempts": 0, "cache_hits": 0, "retries": 0,
                      "failures": 0, "truncated": 0, "timeouts": 0, "seconds": 0.0}

    @property
    def enabled(self) -> bool:
        return self.transport is not None

    # -- public API ----------------------------------------------------------
    def ask_json(self, task: str, system: str, user: str, schema: Dict[str, Any],
                 check: Optional[Callable[[Any], List[str]]] = None,
                 retries: Optional[int] = None) -> Any:
        """Ask for JSON that matches `schema` (and passes the optional `check`)."""
        if not self.enabled:
            raise LLMDisabled("no LLM configured (rules-only mode)")
        key = self._key(task, system, user, schema)
        cached = self._cache_get(key)
        if cached is not None:
            self.stats["cache_hits"] += 1
            return cached
        self.stats["calls"] += 1
        attempts = (self.retries if retries is None else max(0, retries)) + 1
        last_error = "unknown error"
        last_exc: Optional[LLMError] = None
        for attempt in range(attempts):
            prompt = user
            if attempt and isinstance(last_exc, LLMOutputError):
                prompt = (user + "\n\nYour previous answer was rejected: " + last_error +
                          "\nReturn ONLY valid JSON that follows the schema.")
            started = time.time()
            self.stats["attempts"] += 1
            problem = ""
            try:
                reply = self._invoke([("system", system), ("human", prompt)], schema)
                if reply.truncated:
                    self.stats["truncated"] += 1
                    raise LLMOutputError("the answer was cut off at the token limit")
                try:
                    parsed = extract_json(reply.text)
                except ValueError as exc:
                    raise LLMOutputError(str(exc))
                errors = validate_schema(parsed, schema)
                if not errors and check is not None:
                    errors = list(check(parsed))
```

```
                if errors:
                    raise LLMOutputError("; ".join(errors[:5]))
                self._log(task, attempt, started, prompt, reply.text, "")
                self._cache_put(key, parsed)
                self.stats["seconds"] += time.time() - started
                return parsed
            except LLMError as exc:
                last_exc = exc
                last_error = str(exc)
                problem = "%s: %s" % (type(exc).__name__, exc)
                self._log(task, attempt, started, prompt, "", problem)
                self.stats["seconds"] += time.time() - started
                if attempt + 1 < attempts:
                    self.stats["retries"] += 1
                    if not isinstance(exc, LLMOutputError):
                        self.sleep(self.backoff * (2 ** attempt))
        self.stats["failures"] += 1
        raise last_exc if last_exc is not None else LLMOutputError(last_error)

    def ask_text(self, task: str, system: str, user: str, retries: Optional[int] = None) -> str:
        """Ask for free text. Retries on transport problems and empty answers."""
        if not self.enabled:
            raise LLMDisabled("no LLM configured (rules-only mode)")
        key = self._key(task, system, user, None)
        cached = self._cache_get(key)
        if isinstance(cached, str):
            self.stats["cache_hits"] += 1
            return cached
        self.stats["calls"] += 1
        attempts = (self.retries if retries is None else max(0, retries)) + 1
        last_exc: Optional[LLMError] = None
        for attempt in range(attempts):
            started = time.time()
            self.stats["attempts"] += 1
            try:
                reply = self._invoke([("system", system), ("human", user)], None)
                text = reply.text.strip()
                if not text:
                    raise LLMOutputError("empty answer")
                if reply.truncated:
                    self.stats["truncated"] += 1
                self._log(task, attempt, started, user, text, "")
                if not reply.truncated:
                    self._cache_put(key, text)
                self.stats["seconds"] += time.time() - started
                return text
            except LLMError as exc:
                last_exc = exc
                self._log(task, attempt, started, user, "", "%s: %s" % (type(exc).__name__, exc))
                self.stats["seconds"] += time.time() - started
                if attempt + 1 < attempts:
                    self.stats["retries"] += 1
                    self.sleep(self.backoff * (2 ** attempt))
        self.stats["failures"] += 1
        raise last_exc if last_exc is not None else LLMOutputError("no answer")

    # -- internals ------------------------------------------------------------
    def _invoke(self, messages: Messages, schema: Optional[Dict[str, Any]]) -> Reply:
        assert self.transport is not None
        try:
            reply = call_with_timeout(lambda: self.transport(messages, schema), self.timeout)  # type: ignore[misc]
        except LLMTimeout:
            self.stats["timeouts"] += 1
            raise
        except LLMError:
            raise
        except Exception as exc:                        # noqa: BLE001
            raise LLMTransportError("%s: %s" % (type(exc).__name__, exc)) from exc
        if isinstance(reply, str):
            reply = Reply(reply)
        if not isinstance(reply, Reply):
            raise LLMTransportError("transport returned %s, expected Reply" % type(reply).__name__)
        return reply

    def _key(self, task: str, system: str, user: str, schema: Optional[Dict[str, Any]]) -> str:
        blob = json.dumps([self.model_id, self.prompt_version, task, system, user, schema],
                          sort_keys=True, ensure_ascii=False)
        return hashlib.sha256(blob.encode("utf-8")).hexdigest()

    def _cache_get(self, key: str) -> Any:
        if not self.cache_dir:
            return None
        path = os.path.join(self.cache_dir, key + ".json")
        try:
            with open(path, "r", encoding="utf-8") as handle:
                return json.load(handle)["value"]
        except (OSError, ValueError, KeyError):
            return None                                 # missing or corrupt: just recompute

```

```
    def _cache_put(self, key: str, value: Any) -> None:
        if not self.cache_dir:
            return
        try:
            os.makedirs(self.cache_dir, exist_ok=True)
            path = os.path.join(self.cache_dir, key + ".json")
            tmp = path + ".tmp"
            with open(tmp, "w", encoding="utf-8") as handle:
                json.dump({"value": value}, handle, ensure_ascii=False)
            os.replace(tmp, path)
        except OSError:
            pass                                        # cache is best effort

    def _log(self, task: str, attempt: int, started: float, prompt: str, reply: str, error: str) -> None:
        if not self.log_path:
            return
        record: Dict[str, Any] = {"time": time.strftime("%Y-%m-%d %H:%M:%S"), "task": task,
                                  "attempt": attempt, "seconds": round(time.time() - started, 3),
                                  "error": error, "model": self.model_id}
        if self.log_full:
            record["prompt"] = prompt
            record["reply"] = reply
        try:
            os.makedirs(os.path.dirname(self.log_path) or ".", exist_ok=True)
            with open(self.log_path, "a", encoding="utf-8") as handle:
                handle.write(json.dumps(record, ensure_ascii=False) + "\n")
        except OSError:
            pass                                        # logging must never break a run


def build_client(cfg: Dict[str, Any], run_dir: Optional[str], cache_dir: Optional[str] = None) -> LLMClient:
    """Create the client described by cfg['llm']. backend 'none' = rules only."""
    llm = cfg["llm"]
    backend = (llm.get("backend") or "none").lower()
    if backend == "none":
        return LLMClient(None, model_id="none")
    base_url = llm.get("base_url")
    if backend == "ollama" and not base_url:
        base_url = "http://localhost:11434"
    transport = LangChainTransport(
        backend=backend, model=llm["model"], base_url=base_url,
        temperature=float(llm.get("temperature", 0.0)), max_tokens=int(llm.get("max_tokens", 1024)),
        timeout=float(llm.get("timeout", 120.0)), num_ctx=llm.get("num_ctx"),
        fold=bool(llm.get("fold_system", True)))
    use_cache = bool(llm.get("cache", True))
    return LLMClient(
        transport, model_id="%s:%s" % (backend, llm["model"]),
        retries=int(llm.get("retries", 2)), backoff=float(llm.get("backoff", 1.0)),
        timeout=float(llm.get("timeout", 120.0)),
        cache_dir=(cache_dir or (os.path.join(run_dir, "llm_cache") if run_dir else None)) if use_cache else None,
        log_path=os.path.join(run_dir, "llm_log.jsonl") if run_dir else None,
        log_full=bool(llm.get("log_full", True)))
```

`testgen/grounding.py` **(217 lines)**

```
"""Variable grounding: free text -> canonical variable name -> symbol.

The symbol table (symbols.csv) is the single source of truth. The model never
invents or recalls a variable name; it only *chooses* from a short candidate
list that code computed (see choose_with_llm).

Resolution status
    exact      normalized text equals a variable, alias or symbol
    fuzzy      one clear best match (score >= accept and margin >= margin)
    ambiguous  plausible candidates but no clear winner -> ask the LLM / human
    none       nothing close -> flagged, never guessed
"""
from __future__ import annotations

import csv
import io
import os
import re
from difflib import SequenceMatcher
from typing import Any, Dict, List, Optional, Tuple

from .issues import FatalError, make_issue
from .llm_client import LLMClient, LLMError
from .schema import choose_variable_schema

_CAMEL = re.compile(r"([a-z0-9])([A-Z])")
_NON_ALNUM = re.compile(r"[^a-z0-9]+")


def normalize(text: str) -> str:
    """'Driver_Pannel autoGas' -> 'driver pannel auto gas'."""
    text = _CAMEL.sub(r"\1 \2", str(text))
    return _NON_ALNUM.sub(" ", text.lower()).strip()
```

```


def score(a: str, b: str) -> float:
    """Similarity of two *normalized* strings in [0, 1]."""
    if not a or not b:
        return 0.0
    if a == b:
        return 1.0
    ta, tb = set(a.split()), set(b.split())
    token = len(ta & tb) / float(len(ta | tb))
    seq = SequenceMatcher(None, a, b).ratio()
    squashed = SequenceMatcher(None, a.replace(" ", ""), b.replace(" ", "")).ratio()
    return max(seq, squashed, 0.5 * squashed + 0.5 * token)


class SymbolTable:
    """variable -> symbol map with fuzzy lookup."""

    def __init__(self, rows: List[Dict[str, str]]) -> None:
        self.rows: Dict[str, Dict[str, str]] = {}
        self.symbol_to_var: Dict[str, str] = {}
        self._names: Dict[str, List[str]] = {}          # variable -> normalized names
        self._exact: Dict[str, str] = {}                # normalized name -> variable
        self._tokens: Dict[str, set] = {}               # token -> variables
        self._cache: Dict[Tuple[str, int], List[Tuple[str, float]]] = {}
        for row in rows:
            var = row["variable"]
            self.rows[var] = row
            self.symbol_to_var.setdefault(row["symbol"], var)
            names = [var] + [a for a in row.get("aliases", "").split("|") if a.strip()]
            norm_names = []
            for name in names:
                norm = normalize(name)
                if not norm:
                    continue
                norm_names.append(norm)
                self._exact.setdefault(norm, var)
                for tok in norm.split():
                    self._tokens.setdefault(tok, set()).add(var)
            self._names[var] = norm_names
            self._exact.setdefault(normalize(row["symbol"]), var)

    def __len__(self) -> int:
        return len(self.rows)

    def has_variable(self, variable: Optional[str]) -> bool:
        return variable in self.rows

    def symbol_for(self, variable: Optional[str]) -> Optional[str]:
        row = self.rows.get(variable or "")
        return row["symbol"] if row else None

    def description(self, variable: str) -> str:
        return self.rows.get(variable, {}).get("description", "")

    def candidates(self, text: str, top_n: int = 5, floor: float = 0.0) -> List[Tuple[str, float]]:
        """Best-matching variables for `text`, highest score first."""
        norm = normalize(text)
        key = (norm, top_n)
        if key in self._cache:
            return [c for c in self._cache[key] if c[1] >= floor]
        if not norm:
            return []
        pool = set()
        for tok in norm.split():
            pool |= self._tokens.get(tok, set())
        if len(pool) < top_n * 3:                       # few token hits: scan everything
            pool = set(self.rows)
        scored: List[Tuple[str, float]] = []
        for var in pool:
            best = 0.0
            for name in self._names.get(var, []):
                matcher = SequenceMatcher(None, norm, name)
                if matcher.real_quick_ratio() < 0.4 or matcher.quick_ratio() < 0.4:
                    continue
                best = max(best, score(norm, name))
            if best > 0:
                scored.append((var, round(best, 4)))
        scored.sort(key=lambda item: (-item[1], item[0]))
        result = scored[:top_n]
        self._cache[key] = result
        return [c for c in result if c[1] >= floor]

    def resolve(self, text: str, ground_cfg: Dict[str, Any]) -> Dict[str, Any]:
        """Rule-based resolution; see module docstring for the statuses."""
        norm = normalize(text)
        if norm in self._exact:
            var = self._exact[norm]
            return {"status": "exact", "variable": var, "score": 1.0, "candidates": [(var, 1.0)]}
```

```
        cands = self.candidates(text, int(ground_cfg["top_n"]), float(ground_cfg["floor"]))
        if not cands:
            return {"status": "none", "variable": None, "score": 0.0, "candidates": []}
        best = cands[0]
        second = cands[1][1] if len(cands) > 1 else 0.0
        if best[1] >= ground_cfg["accept"] and (best[1] - second) >= ground_cfg["margin"]:
            return {"status": "fuzzy", "variable": best[0], "score": best[1], "candidates": cands}
        return {"status": "ambiguous", "variable": None, "score": best[1], "candidates": cands}


def load_symbols(path: str) -> Tuple[SymbolTable, List[Dict[str, str]]]:
    """Read symbols.csv: columns variable,symbol[,description][,aliases (a|b|c)]."""
    issues: List[Dict[str, str]] = []
    if not path or not os.path.isfile(path):
        raise FatalError("symbols file not found: %s" % path)
    with open(path, "rb") as handle:
        raw = handle.read()
    if not raw.strip():
        raise FatalError("symbols file is empty: %s" % path)
    try:
        text = raw.decode("utf-8-sig")
    except UnicodeDecodeError:
        text = raw.decode("latin-1")
        issues.append(make_issue("info", "config", "symbols file is not UTF-8; read as latin-1"))
    try:
        dialect = csv.Sniffer().sniff(text[:2048], delimiters=",;\t")
    except csv.Error:
        dialect = csv.excel
    reader = csv.DictReader(io.StringIO(text), dialect=dialect)
    if not reader.fieldnames:
        raise FatalError("symbols file has no header row")
    fields = {(name or "").strip().lower(): name for name in reader.fieldnames}
    var_col = fields.get("variable") or fields.get("name")
    sym_col = fields.get("symbol")
    if not var_col or not sym_col:
        raise FatalError("symbols file needs the columns 'variable' and 'symbol' (found: %s)"
                         % ", ".join(reader.fieldnames))
    desc_col = fields.get("description")
    alias_col = fields.get("aliases")
    rows: List[Dict[str, str]] = []
    seen_var: Dict[str, int] = {}
    seen_sym: Dict[str, str] = {}
    for line_no, rec in enumerate(reader, start=2):
        var = (rec.get(var_col) or "").strip()
        sym = (rec.get(sym_col) or "").strip()
        if not var and not sym:
            continue
        if not var or not sym:
            issues.append(make_issue("review", "config", "symbols line %d is incomplete; skipped" % line_no,
                                     "symbols.csv:%d" % line_no, "fill both variable and symbol"))
            continue
        if var in seen_var:
            issues.append(make_issue("review", "config", "duplicate variable %r (line %d); first entry kept"
                                     % (var, line_no), "symbols.csv:%d" % line_no))
            continue
        if sym in seen_sym:
            issues.append(make_issue("review", "config", "symbol %r is used by both %r and %r"
                                     % (sym, seen_sym[sym], var), "symbols.csv:%d" % line_no,
                                     "two variables must not share a symbol"))
        seen_var[var] = line_no
        seen_sym.setdefault(sym, var)
        rows.append({"variable": var, "symbol": sym,
                     "description": (rec.get(desc_col) or "").strip() if desc_col else "",
                     "aliases": (rec.get(alias_col) or "").strip() if alias_col else ""})
    if not rows:
        raise FatalError("symbols file contains no usable rows")
    return SymbolTable(rows), issues


def choose_with_llm(client: LLMClient, symbols: SymbolTable, phrase: str,
                    candidates: List[Tuple[str, float]], context: str,
                    ) -> Optional[str]:
    """Let the model pick ONE of the code-computed candidates (or NONE).

    The answer is constrained to the candidate names by the schema and
    re-checked here; anything else raises LLMError, which the caller turns into
    an 'unresolved' item. Returns the variable or None (model said NONE).
    """
    names = [c[0] for c in candidates]
    if not names:
        return None
    lines = []
    for var, sc in candidates:
        desc = symbols.description(var)
        lines.append("- %s%s" % (var, (" (" + desc + ")") if desc else ""))
    user = (
        "TASK: choose_variable\n"
        "Pick the variable that the phrase refers to. Use only the candidates below. "
        "If none fits, answer NONE.\n\n"
```

```
        "Context: %s\nPhrase: \"%s\"\nCandidates:\n%s\n\n"
        "Answer as JSON: {\"variable\": \"<candidate or NONE>\"}" % (context, phrase, "\n".join(lines)))
    system = "You match phrases from test documents to variable names. Answer only with JSON."
    answer = client.ask_json("choose_variable", system, user, choose_variable_schema(names))
    choice = answer["variable"]
    return None if choice == "NONE" else choice
```

testgen/table_decoder.py **(421 lines)**

```
"""Decode a Word table into plain data, with no LLM.

Why this exists: python-docx `row.cells` repeats merged cells and loses which
value belongs to which header column. Complex tables (several merged header rows,
sub-columns, lane columns 1..4) must be read from the XML grid instead.

Pipeline inside decode_table():
    1. build_grid       XML -> rectangular grid; every cell knows its span and
                        a shared identity across the grid positions it covers
    2. detect_header_rows   how many top rows are headers (overridable)
    3. header paths     each grid column gets a path like
                        ["Section A", "Field 1", "Byte 1", "2"]
    4. fields           columns are grouped: lanes (same parent, numeric last
                        segment, e.g. ... / 1, 2, 3, 4) become ONE field with
                        several values; every other column is a scalar field
    5. rows             each data row becomes {field key: [values]}

Decoded table shape (plain dicts, JSON friendly):
    {"layout_hash": str, "n_header_rows": int, "n_cols": int, "n_rows": int,
     "id_column": bool,
     "fields": [{"key": "Section A / Flag", "path": [...], "kind": "scalar"|"lanes",
                 "lane_labels": ["1","2","3","4"], "cols": [3,4,5,6]}],
     "rows":   [{"row_id": "1", "row_index": 1, "source_row": 4,
                 "values": {"Section A / Flag": ["0"], ...}}],
     "warnings": [str]}
Values are strings (leading zeros are preserved) or None for "don't care".
If you maintain your own decoder, anything that returns this shape can replace
this module.
"""
from __future__ import annotations

import hashlib
import re
from typing import Any, Dict, Iterator, List, Optional, Sequence

from docx.oxml.ns import qn

SEP = " / "
MAX_COLS = 400
MAX_ROWS = 5000
ID_HEADER_RE = re.compile(
    r"^(row|no\.?|#|tc|id|tc\s*id|test\s*(case|step|id|no\.?)|step|seq\.?|index|s\.?\s*no\.?|sl\.?\s*no\.?)$",
    re.I)
_NUMERIC_RE = re.compile(r"^-?\d+(\.\d+)?$")
_DIGITS_RE = re.compile(r"^\d+$")

Cell = Dict[str, Any]
Grid = List[List[Cell]]


# ---------------------------------------------------------------------------
# 1. XML -> grid
# ---------------------------------------------------------------------------
def _cell_text(tc: Any) -> str:
    """All text of a cell (nested tables flattened), whitespace collapsed."""
    parts = []
    for para in tc.iter(qn("w:p")):
        text = "".join(t.text or "" for t in para.iter(qn("w:t")))
        text = re.sub(r"\s+", " ", text).strip()
        if text:
            parts.append(text)
    return " ".join(parts)


def _int_val(element: Any, default: int) -> int:
    if element is None:
        return default
    try:
        return int(element.get(qn("w:val")))
    except (TypeError, ValueError):
        return default


def _tr_cells(tr: Any) -> Iterator[Any]:
    """Cells of a row, also those wrapped in content controls (w:sdt)."""
    for child in tr:
        if child.tag == qn("w:tc"):
            yield child
        elif child.tag == qn("w:sdt"):
```

```
            content = child.find(qn("w:sdtContent"))
            if content is not None:
                for inner in content:
                    if inner.tag == qn("w:tc"):
                        yield inner


def build_grid(tbl: Any, warnings: List[str]) -> Grid:
    """Rectangular grid of cell dicts. Merged cells share ONE dict object."""
    rows = list(tbl.findall(qn("w:tr")))
    if not rows:
        return []
    next_id = [0]

    def new_cell(text: str, r: int, c: int, span: int) -> Cell:
        next_id[0] += 1
        return {"id": next_id[0], "text": text, "r0": r, "c0": c, "colspan": span, "rowspan": 1}

    grid: Grid = []
    open_v: Dict[int, Cell] = {}           # column -> cell whose vertical merge is open
    for r_idx, tr in enumerate(rows):
        row: List[Cell] = []
        tr_pr = tr.find(qn("w:trPr"))
        if tr_pr is not None:
            for _ in range(max(0, min(_int_val(tr_pr.find(qn("w:gridBefore")), 0), MAX_COLS))):
                row.append(new_cell("", r_idx, len(row), 1))
        new_open: Dict[int, Cell] = {}
        for tc in _tr_cells(tr):
            tc_pr = tc.find(qn("w:tcPr"))
            span = 1
            v_merge = None
            if tc_pr is not None:
                span = max(1, min(_int_val(tc_pr.find(qn("w:gridSpan")), 1), MAX_COLS))
                v_merge = tc_pr.find(qn("w:vMerge"))
            col = len(row)
            cell: Optional[Cell] = None
            if v_merge is not None and v_merge.get(qn("w:val")) in (None, "continue"):
                origin = open_v.get(col)
                if origin is not None and origin["colspan"] == span and origin["c0"] == col:
                    origin["rowspan"] += 1
                    cell = origin
                else:
                    warnings.append("row %d: vertical merge without a start cell; treated as a normal cell"
                                    % (r_idx + 1))
            if cell is None:
                cell = new_cell(_cell_text(tc), r_idx, col, span)
            if v_merge is not None:
                for k in range(span):
                    new_open[col + k] = cell
            for _ in range(span):
                row.append(cell)
            if len(row) > MAX_COLS:
                raise ValueError("table has more than %d grid columns" % MAX_COLS)
        open_v = new_open
        grid.append(row)
    width = max(len(r) for r in grid)
    if width == 0:
        return []
    for r_idx, row in enumerate(grid):
        while len(row) < width:
            row.append(new_cell("", r_idx, len(row), 1))
    if len(grid) > MAX_ROWS:
        warnings.append("table has %d rows; only the first %d were read" % (len(grid), MAX_ROWS))
        grid = grid[:MAX_ROWS]
    return grid


def transpose_grid(grid: Grid) -> Grid:
    """Swap rows and columns (for tables where each test case is a column)."""
    if not grid:
        return grid
    mapping: Dict[int, Cell] = {}
    for row in grid:
        for cell in row:
            if cell["id"] not in mapping:
                mapping[cell["id"]] = {"id": cell["id"], "text": cell["text"], "r0": cell["c0"],
                                       "c0": cell["r0"], "rowspan": cell["colspan"],
                                       "colspan": cell["rowspan"]}
    n_rows, n_cols = len(grid), len(grid[0])
    return [[mapping[grid[r][c]["id"]] for r in range(n_rows)] for c in range(n_cols)]


# ---------------------------------------------------------------------------
# 2. header rows
# ---------------------------------------------------------------------------
def unique_cells(row: Sequence[Cell]) -> List[Cell]:
    seen = set()
    out = []
    for cell in row:
```

```
        if cell["id"] not in seen:
            seen.add(cell["id"])
            out.append(cell)
    return out


def _has_hspan(row: Sequence[Cell]) -> bool:
    return any(c["colspan"] > 1 and c["text"] for c in unique_cells(row))


def _is_counter_row(row: Sequence[Cell]) -> bool:
    """A row like 1 2 3 4 | 1 2 3 4 (lane numbers under a group header)."""
    texts = [c["text"] for c in unique_cells(row) if c["text"]]
    if len(texts) < 2 or not all(_DIGITS_RE.match(t) for t in texts) or texts[0] != "1":
        return False
    prev = 1
    for text in texts[1:]:
        value = int(text)
        if value != prev + 1 and value != 1:
            return False
        prev = value
    return True


def _mostly_numeric(row: Sequence[Cell]) -> bool:
    texts = [c["text"] for c in unique_cells(row) if c["text"]]
    if not texts:
        return False
    return sum(1 for t in texts if _NUMERIC_RE.match(t)) / float(len(texts)) >= 0.7


def detect_header_rows(grid: Grid) -> int:
    """Number of header rows (at least 1). Rules, in order, applied row by row:
      a) the row continues a vertical merge that started in a header row
      b) the row is a lane-counter row (1 2 3 4 ...) under a spanned header
      c) the row above has horizontally merged cells and this row has more
         cells, mostly text (sub-headers)
    Override with table_options.header_rows when a document defeats these rules.
    """
    n = len(grid)
    if n == 0:
        return 0
    k = 1
    while k < n:
        cur, prev = grid[k], grid[k - 1]
        if any(c["r0"] < k < c["r0"] + c["rowspan"] for c in unique_cells(cur)):
            k += 1
            continue
        if _has_hspan(prev) and _is_counter_row(cur):
            k += 1
            continue
        if (_has_hspan(prev) and len(unique_cells(cur)) > len(unique_cells(prev))
                and not _mostly_numeric(cur)
                and sum(1 for c in unique_cells(cur) if c["text"]) >= 0.5 * len(unique_cells(cur))):
            k += 1
            continue
        break
    return k


# ---------------------------------------------------------------------------
# 3-5. paths, fields, rows
# ---------------------------------------------------------------------------
def header_path(grid: Grid, col: int, n_header: int) -> List[str]:
    path: List[str] = []
    last_id = None
    for r in range(n_header):
        cell = grid[r][col]
        if cell["id"] == last_id:
            continue
        last_id = cell["id"]
        text = cell["text"]
        if text and (not path or path[-1] != text):
            path.append(text)
    return path


def _unique_key(key: str, used: set) -> str:
    if key not in used:
        used.add(key)
        return key
    n = 2
    while "%s [%d]" % (key, n) in used:
        n += 1
    key = "%s [%d]" % (key, n)
    used.add(key)
    return key


```

```
def _looks_like_id_column(grid: Grid, n_header: int, path: List[str], opt: Optional[bool]) -> bool:
    if opt is not None:
        return bool(opt)
    data = [grid[r][0] for r in range(n_header, len(grid))]
    leaf = path[-1] if path else ""
    if len(path) >= 2 and _DIGITS_RE.match(leaf):
        return False
    if ID_HEADER_RE.match(leaf.strip()):
        return True
    texts = [c["text"] for c in data]
    if len(texts) >= 2 and all(_DIGITS_RE.match(t) for t in texts):
        return [int(t) for t in texts] == list(range(1, len(texts) + 1))
    return False


def decode_table(tbl: Any, options: Optional[Dict[str, Any]] = None,
                 dont_care: Sequence[str] = ("", "-", "x", "n/a")) -> Dict[str, Any]:
    """Decode one <w:tbl> element. Raises ValueError for unusable tables."""
    opts = options or {}
    warnings: List[str] = []
    grid = build_grid(tbl, warnings)
    if not grid:
        raise ValueError("table has no cells")
    if opts.get("transpose"):
        grid = transpose_grid(grid)
    width = len(grid[0])
    n_rows = len(grid)
    override = opts.get("header_rows")
    if isinstance(override, int) and not isinstance(override, bool) and override >= 1:
        n_header = min(override, n_rows)
        if override > n_rows:
            warnings.append("header_rows=%d is larger than the table (%d rows)" % (override, n_rows))
    else:
        n_header = detect_header_rows(grid)
    paths = [header_path(grid, c, n_header) for c in range(width)]
    has_id = width >= 2 and _looks_like_id_column(grid, n_header, paths[0], opts.get("id_column"))
    first_col = 1 if has_id else 0

    # --- fields: group lane columns, everything else is scalar -----------------
    care = {str(v).strip().lower() for v in dont_care}
    fields: List[Dict[str, Any]] = []
    used: set = set()
    col = first_col
    while col < width:
        path = paths[col]
        lane_leaf = len(path) >= 2 and _DIGITS_RE.match(path[-1])
        if lane_leaf:
            parent = path[:-1]
            cols = [col]
            labels = [path[-1]]
            nxt = col + 1
            while (nxt < width and len(paths[nxt]) >= 2 and paths[nxt][:-1] == parent
                   and _DIGITS_RE.match(paths[nxt][-1]) and int(paths[nxt][-1]) > int(labels[-1])):
                cols.append(nxt)
                labels.append(paths[nxt][-1])
                nxt += 1
            if len(cols) >= 2:
                fields.append({"key": _unique_key(SEP.join(parent), used), "path": list(parent),
                               "kind": "lanes", "lane_labels": labels, "cols": cols})
                col = nxt
                continue
        key = SEP.join(path) if path else "col%d" % (col + 1)
        if not path:
            warnings.append("column %d has no header text; named %r" % (col + 1, key))
        fields.append({"key": _unique_key(key, used), "path": list(path), "kind": "scalar",
                       "lane_labels": [], "cols": [col]})
        col += 1

    # --- data rows -------------------------------------------------------------
    rows: List[Dict[str, Any]] = []
    blank = 0
    merged_note = False
    for r in range(n_header, n_rows):
        values: Dict[str, List[Optional[str]]] = {}
        any_value = False
        for fld in fields:
            vals: List[Optional[str]] = []
            for c in fld["cols"]:
                cell = grid[r][c]
                if cell["colspan"] > 1 or cell["rowspan"] > 1:
                    merged_note = True
                text = cell["text"].strip()
                if text.lower() in care:
                    vals.append(None)
                else:
                    vals.append(text)
                    any_value = True
            values[fld["key"]] = vals
        row_id = grid[r][0]["text"].strip() if has_id else ""
```

```
        if not any_value and not row_id:
            blank += 1
            continue
        index = len(rows) + 1
        rows.append({"row_id": row_id or str(index), "row_index": index,
                     "source_row": r, "values": values})
    if blank:
        warnings.append("%d blank row(s) skipped" % blank)
    if merged_note:
        warnings.append("data cells are merged; a merged value is repeated for every column it covers")
    if not rows:
        warnings.append("table has a header but no data rows")
    layout = "|".join("%s:%s:%d" % (f["key"], f["kind"], len(f["cols"])) for f in fields) + "|h%d" % n_header
    return {"layout_hash": hashlib.sha1(layout.encode("utf-8")).hexdigest()[:12],
            "n_header_rows": n_header, "n_cols": width, "n_rows": len(rows),
            "id_column": has_id, "fields": fields, "rows": rows, "warnings": warnings}


# ---------------------------------------------------------------------------
# text views (for review files and fallbacks; the pipeline itself never needs them)
# ---------------------------------------------------------------------------
def _cell(value: Optional[str]) -> str:
    return "" if value is None else str(value).replace("|", "\\|")


def to_markdown(decoded: Dict[str, Any], style: str = "per_case") -> str:
    """per_case: one small table per row (best for a human or a small model).
    flat: one wide table (compact, hard to read when there are many columns)."""
    fields, rows = decoded["fields"], decoded["rows"]
    out: List[str] = []
    if style == "flat":
        heads = []
        for f in fields:
            if f["kind"] == "lanes":
                heads.extend("%s #%s" % (f["key"], lab) for lab in f["lane_labels"])
            else:
                heads.append(f["key"])
        out.append("| Row | " + " | ".join(_cell(h) for h in heads) + " |")
        out.append("|" + "---|" * (len(heads) + 1))
        for row in rows:
            cells: List[str] = []
            for f in fields:
                cells.extend(_cell(v) for v in row["values"][f["key"]])
            out.append("| %s | %s |" % (_cell(row["row_id"]), " | ".join(cells)))
        return "\n".join(out)
    max_lanes = max([len(f["lane_labels"]) for f in fields if f["kind"] == "lanes"] or [0])
    head = ["Field", "Value"] + ["Lane %d" % (i + 1) for i in range(max_lanes)]
    for row in rows:
        out.append("### Row %s" % row["row_id"])
        out.append("| " + " | ".join(head) + " |")
        out.append("|" + "---|" * len(head))
        for f in fields:
            vals = row["values"][f["key"]]
            if f["kind"] == "lanes":
                lanes = [_cell(v) for v in vals] + [""] * (max_lanes - len(vals))
                out.append("| %s |  | %s |" % (_cell(f["key"]), " | ".join(lanes)))
            else:
                out.append("| %s | %s |%s" % (_cell(f["key"]), _cell(vals[0]), " |" * max_lanes))
        out.append("")
    return "\n".join(out)


def flatten_text(decoded: Dict[str, Any]) -> str:
    """One line per row: 'Row 1: key=value; key=[a,b,c,d]'."""
    lines = []
    for row in decoded["rows"]:
        parts = []
        for f in decoded["fields"]:
            vals = row["values"][f["key"]]
            if f["kind"] == "lanes":
                parts.append("%s=[%s]" % (f["key"], ",".join("" if v is None else v for v in vals)))
            else:
                parts.append("%s=%s" % (f["key"], "" if vals[0] is None else vals[0]))
        lines.append("Row %s: %s" % (row["row_id"], "; ".join(parts)))
    return "\n".join(lines)
```

testgen/docx_loader.py **(182 lines)**

```
"""Stage 0: read the .docx in document order and produce blocks.

A *block* is one heading, paragraph, list item or table:
    {"id": "B001", "order": 1, "kind": "heading"|"paragraph"|"list"|"table",
     "text": str, "level": int, "style": str, "heading_source": "style"|"heuristic"|"",
     "table_index": int|None, "table": decoded-table dict | None}

Robustness rules
    - not a .docx / corrupt zip / unreadable  -> FatalError with a clear message
    - no content at all                       -> FatalError
```

```
    - a table that cannot be decoded          -> block kept with a flattened text,
                                                 review issue, run continues
    - headings come from styles or outline levels; only if the whole document has
      none, bold/numbered short paragraphs are promoted (and an issue says so)
"""
from __future__ import annotations

import os
import re
from typing import Any, Dict, Iterator, List, Tuple

from docx import Document
from docx.oxml.ns import qn
from docx.text.paragraph import Paragraph

from .issues import FatalError, make_issue
from .table_decoder import decode_table, flatten_text

_NUMBERED_RE = re.compile(r"^\d+(\.\d+){0,4}[.)]?\s+\S")


def _iter_body(parent: Any) -> Iterator[Tuple[str, Any]]:
    """Yield ('p'|'tbl', element) in document order, looking inside content controls."""
    for child in parent:
        if child.tag == qn("w:p"):
            yield "p", child
        elif child.tag == qn("w:tbl"):
            yield "tbl", child
        elif child.tag == qn("w:sdt"):
            content = child.find(qn("w:sdtContent"))
            if content is not None:
                for item in _iter_body(content):
                    yield item


def _p_text(p_el: Any) -> str:
    text = "".join(t.text or "" for t in p_el.iter(qn("w:t")))
    return re.sub(r"\s+", " ", text).strip()


def _style_name(para: Paragraph) -> str:
    try:
        return (para.style.name or "").strip()
    except Exception:                                  # noqa: BLE001 - broken style table
        return ""


def _outline_level(p_el: Any) -> int:
    p_pr = p_el.find(qn("w:pPr"))
    if p_pr is None:
        return -1
    lvl = p_pr.find(qn("w:outlineLvl"))
    if lvl is None:
        return -1
    try:
        return int(lvl.get(qn("w:val")))
    except (TypeError, ValueError):
        return -1


def _is_list(p_el: Any, style: str) -> bool:
    p_pr = p_el.find(qn("w:pPr"))
    has_num = p_pr is not None and p_pr.find(qn("w:numPr")) is not None
    return has_num or style.lower().startswith(("list", "toc"))


def _all_bold(para: Paragraph) -> bool:
    runs = [r for r in para.runs if r.text.strip()]
    return bool(runs) and all(bool(r.bold) for r in runs)


def load_blocks(path: str, cfg: Dict[str, Any]) -> Tuple[List[Dict[str, Any]], List[Dict[str, str]]]:
    """Return (blocks, issues). Raises FatalError for unusable input."""
    issues: List[Dict[str, str]] = []
    if not path or not os.path.isfile(path):
        raise FatalError("input file not found: %s" % path)
    ext = os.path.splitext(path)[1].lower()
    if ext == ".doc":
        raise FatalError("old .doc format is not supported; save the file as .docx first")
    if ext not in (".docx", ".docm", ".dotx"):
        raise FatalError("expected a .docx file, got %r" % ext)
    if os.path.getsize(path) == 0:
        raise FatalError("input file is empty: %s" % path)
    try:
        doc = Document(path)
    except Exception as exc:                           # noqa: BLE001 - python-docx raises many types
        raise FatalError("cannot open %s as a Word document (%s: %s)"
                         % (os.path.basename(path), type(exc).__name__, exc))

```

```
    raw: List[Dict[str, Any]] = []
    opts = cfg["table_options"]
    dont_care = cfg["dont_care_values"]
    table_no = 0
    for kind, el in _iter_body(doc.element.body):
        if kind == "p":
            para = Paragraph(el, doc)
            text = _p_text(el)
            if not text:
                continue
            style = _style_name(para)
            low = style.lower()
            level = 0
            heading_source = ""
            block_kind = "paragraph"
            if low.startswith("heading") or low in ("title", "toc heading"):
                match = re.search(r"(\d+)\s*$", style)
                level = int(match.group(1)) if match else 1
                block_kind, heading_source = "heading", "style"
            elif _outline_level(el) >= 0:
                level = _outline_level(el) + 1
                block_kind, heading_source = "heading", "style"
            elif _is_list(el, style):
                block_kind = "list"
            raw.append({"kind": block_kind, "text": text, "level": level, "style": style,
                        "heading_source": heading_source, "table_index": None, "table": None,
                        "_para": para})
        else:
            table_no += 1
            per = opts.get("per_table", {}).get(str(table_no - 1), {})
            merged = dict(opts)
            merged.update({k: v for k, v in per.items()})
            decoded = None
            text = ""
            try:
                decoded = decode_table(el, merged, dont_care)
                text = flatten_text(decoded)
            except Exception as exc:                    # noqa: BLE001 - any failure is non-fatal here
                from docx.table import Table
                try:
                    rows = [[c.text.strip() for c in r.cells] for r in Table(el, doc).rows]
                    text = "\n".join(" | ".join(r) for r in rows)
                except Exception:                       # noqa: BLE001
                    text = ""
                issues.append(make_issue(
                    "review", "load", "table %d could not be decoded: %s" % (table_no, exc),
                    "table %d" % table_no, "fix the table or set table_options.per_table in config.json"))
            for warn in (decoded or {}).get("warnings", []):
                issues.append(make_issue("info", "load", "table %d: %s" % (table_no, warn), "table %d" % table_no))
            if decoded is not None and not decoded["fields"]:
                decoded = None
            raw.append({"kind": "table", "text": text, "level": 0, "style": "", "heading_source": "",
                        "table_index": table_no - 1, "table": decoded, "_para": None})

    if not raw:
        raise FatalError("the document has no text or tables: %s" % os.path.basename(path))

    # Heuristic headings only when the document has no real headings at all.
    if not any(b["kind"] == "heading" for b in raw):
        promoted = 0
        for b in raw:
            if b["kind"] != "paragraph" or len(b["text"]) > 80 or b["text"].endswith((".", ",", ";")):
                continue
            if _all_bold(b["_para"]) or _NUMBERED_RE.match(b["text"]):
                dots = b["text"].split(" ")[0].count(".")
                b["kind"], b["level"], b["heading_source"] = "heading", min(dots + 1, 4), "heuristic"
                promoted += 1
        if promoted:
            issues.append(make_issue("info", "load",
                                     "no heading styles found; %d short bold/numbered paragraphs were treated as headi
      +ngs" % promoted,
                                     fix="use Word heading styles for more reliable sections"))
        else:
            issues.append(make_issue("review", "load", "document has no headings; sections cannot be found by title",
                                     fix="the LLM will label the whole document as one section"))

    width = max(3, len(str(len(raw))))
    blocks: List[Dict[str, Any]] = []
    for i, b in enumerate(raw, start=1):
        b.pop("_para", None)
        b["id"] = "B%0*d" % (width, i)
        b["order"] = i
        blocks.append(b)
    return blocks, issues
```

testgen/prompts.py **(102 lines)**

```
"""All prompt text in one place.
```

```
 2
 3  Design rules for a small (9B, 4-bit) model
 4      - one narrow task per call, answer is a short JSON object
 5      - the first line is 'TASK: <name>' (also lets tests route fake answers)
 6      - labels/variables are chosen from lists given in the prompt, never invented
 7      - 'reason' is written before the answer so the model thinks first
 8      - no long rule lists; knowledge that can be written as code lives in code
 9  Bump llm_client.PROMPT_VERSION after editing any text here.
10  """
11  from __future__ import annotations
12
13  from typing import Dict, List, Sequence
14
15  SYSTEM_JSON = "You analyse software test description documents. Answer only with JSON."
16
17  SECTION_LABEL_HELP = {
18      "index": "table of contents or list of chapters",
19      "document_info": "authors, revision history, scope, introduction, glossary",
20      "requirements": "requirement descriptions",
21      "initialization": "system or test initialization, preconditions, boot, global setup",
22      "test_inputs": "test cases or steps with input values to set",
23      "expected_outputs": "expected results or outputs to verify",
24      "constraints": "constraints, notes, assumptions, limitations",
25      "other": "anything else",
26  }
27
28
29  def section_labels_prompt(items: Sequence[Dict[str, str]], labels: Sequence[str]) -> str:
30      """items: [{"id","heading","summary","preview","hint"}]"""
31      label_lines = "\n".join("- %s: %s" % (l, SECTION_LABEL_HELP.get(l, "")) for l in labels)
32      sec_lines = []
33      for it in items:
34          sec_lines.append('[%s] heading: "%s" | %s | hint: %s\n    preview: %s'
35                           % (it["id"], it["heading"], it["summary"], it["hint"] or "none", it["preview"]))
36      ids = ", ".join(it["id"] for it in items)
37      return (
38          "TASK: section_labels\n"
39          "Give every section of a test description document exactly one label.\n"
40          "Labels:\n%s\n\n"
41          "Decide from the heading first, then the content. The hint comes from a keyword rule and may be wrong.\n\n"
42          "Sections:\n%s\n\n"
43          "Answer as JSON with one entry per section (%s):\n"
44          '{"labels": [{"id": "S001", "label": "index"}]}' % (label_lines, "\n".join(sec_lines), ids))
45
46
47  def table_role_prompt(heading: str, caption: str, section_label: str, headers: Sequence[str],
48                        first_row: str, hint: str, n_rows: int) -> str:
49      return (
50          "TASK: table_role\n"
51          "Decide what this table is for in a test description.\n"
52          "- input: rows give values to SET (stimuli) for each test step\n"
53          "- expected: rows give values that must be CHECKED after each test step\n"
54          "- mixed: the same rows contain both inputs and expected values\n"
55          "- ignore: anything else (authors, requirements, history, notes)\n\n"
56          "Section heading: %s\nSection label: %s\nText just before the table: %s\n"
57          "Rule-based hint: %s\nData rows: %d\nColumn headers: %s\nFirst row: %s\n\n"
58          'Answer as JSON: {"reason": "<one short sentence>", "role": "<input|expected|mixed|ignore>"}'
59          % (heading or "(none)", section_label, caption or "(none)", hint or "none", n_rows,
60             "; ".join(headers) if headers else "(none)", first_row or "(empty)"))
61
62
63  def field_map_prompt(role: str, field_key: str, kind: str, lane_labels: Sequence[str],
64                       samples: Sequence[str], candidates: Sequence[str], kinds: Dict[str, str]) -> str:
65      cand = "\n".join("- %s" % c for c in candidates) or "- (no candidates)"
66      kind_lines = "\n".join("- %s: %s" % (k, v) for k, v in kinds.items())
67      lanes = (" with sub-columns %s" % ", ".join(lane_labels)) if lane_labels else ""
68      return (
69          "TASK: field_map\n"
70          "A table column of a test description must be linked to a variable.\n"
71          "Table role: %s\nColumn header path: %s%s\nExample values: %s\n\n"
72          "Candidate variables:\n%s\n\nKinds:\n%s\n\n"
73          "Choose one candidate variable (or NONE if the column is not a variable, e.g. a remark) and a kind.\n"
74          'Answer as JSON: {"reason": "<one short sentence>", "variable": "<candidate or NONE>", "kind": "<kind>"}'
75          % (role, field_key, lanes, "; ".join(samples) or "(empty)", cand, kind_lines))
76
77
78  NOTES_SYSTEM = "You read short parts of a test description and extract facts. Do not invent anything."
79
80
81  def notes_normalize_prompt(text: str, known_funcs: Sequence[str]) -> str:
82      funcs = ", ".join(known_funcs) if known_funcs else "(not restricted)"
83      return (
84          "TASK: notes_normalize\n"
85          "Rewrite the text below as short lines under these headings. Write NONE under a heading if nothing applies.\n"
86          "FUNCTIONS: functions to run (known functions: %s)\n"
87          "CHECKS: variable must equal a value\n"
88          "VERIFY: variable to be looked at manually\n"
89          "CONSTRAINTS: rules or limits for the tests\n"
90          "Copy variable names and values exactly as written. Do not add anything.\n\n"
```

```
        "TEXT:\n%s" % (funcs, text))


def notes_extract_prompt(text: str, normalized: str) -> str:
    return (
        "TASK: notes_extract\n"
        "Fill the JSON from the notes. 'evidence' must be a short quote copied exactly from the ORIGINAL TEXT.\n"
        "Copy variable names exactly as written there.\n\n"
        "ORIGINAL TEXT:\n%s\n\nNOTES:\n%s\n\n"
        'Answer as JSON: {"funcs": [], "test_variables": [{"evidence": "", "variable": "", "expected_value": ""}], '
        '"verify_variables": [{"evidence": "", "variable": ""}], "constraints": [{"evidence": "", "text": ""}]}'
        % (text, normalized))
```

testgen/sections.py **(179 lines)**

```
"""Stage 1: group blocks into sections and label every section.

Section = one heading plus the blocks until the next heading. Labels:
index, document_info, requirements, initialization, test_inputs,
expected_outputs, constraints, other.

Labelling order
    1. a keyword rule gives a *hint* (cheap, deterministic)
    2. the LLM labels all sections (enum-constrained, in batches with overlap
       when the document has many sections); disagreements with the rule are
       reported
    3. if the LLM is off or fails, the rule label is used; with no rule match the
       label is 'other' and a review issue is raised
"""
from __future__ import annotations

from typing import Any, Dict, List, Optional, Tuple

from .config import SECTION_LABELS
from .grounding import normalize
from .issues import make_issue
from .llm_client import LLMClient, LLMError
from .prompts import SYSTEM_JSON, section_labels_prompt
from .schema import section_labels_schema

PRIORITY = ["expected_outputs", "test_inputs", "initialization", "constraints",
            "requirements", "document_info", "index"]
USEFUL = ("initialization", "test_inputs", "expected_outputs", "constraints")


def build_sections(blocks: List[Dict[str, Any]], preview_chars: int = 300) -> List[Dict[str, Any]]:
    """Cut the block list at every heading."""
    sections: List[Dict[str, Any]] = []
    current: Optional[Dict[str, Any]] = None

    def new(heading: str, level: int, order: int) -> Dict[str, Any]:
        sec = {"id": "S%03d" % len(sections), "heading": heading, "level": level, "start_order": order,
               "block_ids": [], "table_block_ids": [], "n_paragraphs": 0, "n_lists": 0,
               "label": None, "rule_label": None, "label_source": "", "preview": ""}
        sections.append(sec)
        return sec

    for b in blocks:
        if b["kind"] == "heading":
            current = new(b["text"], b["level"], b["order"])
            continue
        if current is None:
            current = new("(preamble)", 0, b["order"])
        current["block_ids"].append(b["id"])
        if b["kind"] == "table":
            current["table_block_ids"].append(b["id"])
        elif b["kind"] == "list":
            current["n_lists"] += 1
        else:
            current["n_paragraphs"] += 1
    by_id = {b["id"]: b for b in blocks}
    for sec in sections:
        pieces = []
        for bid in sec["block_ids"]:
            b = by_id[bid]
            if b["kind"] == "table":
                heads = [f["key"] for f in (b["table"] or {}).get("fields", [])][:6]
                pieces.append("[table: %s]" % (", ".join(heads) if heads else "unreadable"))
            else:
                pieces.append(b["text"])
        sec["preview"] = " | ".join(pieces)[:preview_chars]
    return sections


def rule_label(text: str, rules: Dict[str, List[str]]) -> Optional[str]:
    """Best keyword-rule label for a heading, or None."""
    norm = " " + normalize(text) + " "
    best: Optional[Tuple[int, int, str]] = None
```

```
    for label, words in rules.items():
        hits = sum(1 for w in words if normalize(w) and normalize(w) in norm)
        if not hits:
            continue
        prio = PRIORITY.index(label) if label in PRIORITY else len(PRIORITY)
        cand = (hits, -prio, label)
        if best is None or cand > best:
            best = cand
    return best[2] if best else None


def plan_batches(n: int, size: int, overlap: int) -> List[List[int]]:
    """Index windows of `size` sections that overlap by `overlap`."""
    size = max(1, size)
    overlap = max(0, min(overlap, size - 1))
    if n <= size:
        return [list(range(n))] if n else []
    batches, start = [], 0
    while True:
        end = min(start + size, n)
        batches.append(list(range(start, end)))
        if end >= n:
            break
        start = end - overlap
    return batches


def _summary(sec: Dict[str, Any]) -> str:
    parts = []
    if sec["n_paragraphs"]:
        parts.append("%d paragraph(s)" % sec["n_paragraphs"])
    if sec["n_lists"]:
        parts.append("%d list item(s)" % sec["n_lists"])
    if sec["table_block_ids"]:
        parts.append("%d table(s)" % len(sec["table_block_ids"]))
    return ", ".join(parts) or "empty"


def label_sections(sections: List[Dict[str, Any]], client: LLMClient, cfg: Dict[str, Any]
                   ) -> List[Dict[str, str]]:
    """Fill sec['label'] / ['rule_label'] / ['label_source'] in place; return issues."""
    issues: List[Dict[str, str]] = []
    rules = cfg["keyword_rules"]
    scfg = cfg["sections"]
    for sec in sections:
        sec["rule_label"] = rule_label(sec["heading"], rules)
    llm_labels: Dict[str, str] = {}
    if client.enabled and sections:
        for batch in plan_batches(len(sections), int(scfg["batch_size"]), int(scfg["overlap"])):
            sub = [sections[i] for i in batch]
            preview = int(scfg["preview_chars"])
            while True:                                     # shrink previews to fit the prompt budget
                items = [{"id": s["id"], "heading": s["heading"], "summary": _summary(s),
                          "preview": s["preview"][:preview], "hint": s["rule_label"] or ""} for s in sub]
                prompt = section_labels_prompt(items, SECTION_LABELS)
                if len(prompt) <= int(scfg["max_prompt_chars"]) or preview <= 40:
                    break
                preview = max(40, preview // 2)
            ids = [s["id"] for s in sub]

            def check(parsed: Any, ids: List[str] = ids) -> List[str]:
                got = [e["id"] for e in parsed["labels"]]
                missing = [i for i in ids if i not in got]
                dup = sorted({i for i in got if got.count(i) > 1})
                errs = []
                if missing:
                    errs.append("missing sections %s" % missing)
                if dup:
                    errs.append("sections labelled twice: %s" % dup)
                return errs
            try:
                answer = client.ask_json("section_labels", SYSTEM_JSON, prompt,
                                         section_labels_schema(ids, SECTION_LABELS), check=check)
            except LLMError as exc:
                issues.append(make_issue("review", "sections",
                                         "LLM labelling failed for sections %s..%s (%s); rule labels used"
                                         % (ids[0], ids[-1], exc), "%s..%s" % (ids[0], ids[-1])))
                continue
            for entry in answer["labels"]:
                prev = llm_labels.get(entry["id"])
                if prev is None:
                    llm_labels[entry["id"]] = entry["label"]
                elif prev != entry["label"]:
                    issues.append(make_issue("review", "sections",
                                             "overlapping batches disagree on %s (%s vs %s); first kept"
                                             % (entry["id"], prev, entry["label"]), entry["id"]))
    for sec in sections:
        llm = llm_labels.get(sec["id"])
        rule = sec["rule_label"]
```

```
        if llm:
            sec["label"], sec["label_source"] = llm, "llm"
            if rule and rule != llm:
                sev = "review" if (llm in USEFUL or rule in USEFUL) else "info"
                issues.append(make_issue(sev, "sections",
                                         "label %r (LLM) differs from keyword rule %r for heading %r"
                                         % (llm, rule, sec["heading"]), sec["id"],
                                         "check the section label in 01_sections.json"))
        elif rule:
            sec["label"], sec["label_source"] = rule, "rule"
        else:
            sec["label"], sec["label_source"] = "other", "default"
            if sec["block_ids"]:
                issues.append(make_issue("review", "sections",
                                         "could not label section %r; treated as 'other'" % sec["heading"],
                                         sec["id"], "set the label by hand in 01_sections.json and resume from stage 2
      +"))
    return issues
```

testgen/roles.py **(122 lines)**

```
"""Stage 2: decide what each table is (input, expected, mixed, ignore).

Decision order per table
    1. section label: test_inputs -> input, expected_outputs -> expected
    2. a section with several data tables, or a heading/caption that contradicts
       the section label, is ambiguous -> ask the LLM (enum), else use keywords
    3. tables in sections labelled 'other' use keywords, then the LLM
    4. tables in index/document_info/requirements/constraints/initialization
       sections are ignored (the initialization file is handled elsewhere)
At least one input or expected table must exist, otherwise an error is raised.
"""
from __future__ import annotations

import re
from typing import Any, Dict, List, Optional, Tuple

from .grounding import normalize
from .issues import make_issue
from .llm_client import LLMClient, LLMError
from .prompts import SYSTEM_JSON, table_role_prompt
from .schema import table_role_schema

ROLES = ["input", "expected", "mixed", "ignore"]
INPUT_WORDS = ["input", "stimul", "set value", "setting", "test step", "test case", "precondition"]
EXPECTED_WORDS = ["expected", "output", "result", "verify", "check", "response", "outcome"]


def keyword_role(text: str) -> Optional[str]:
    """'input' / 'expected' from keywords, None if neither or both match equally."""
    norm = " " + normalize(text) + " "
    n_in = sum(1 for w in INPUT_WORDS if w in norm)
    n_out = sum(1 for w in EXPECTED_WORDS if w in norm)
    if n_in > n_out:
        return "input"
    if n_out > n_in:
        return "expected"
    return None


def _caption(blocks_by_id: Dict[str, Dict[str, Any]], section: Dict[str, Any], table_id: str) -> str:
    ids = section["block_ids"]
    pos = ids.index(table_id)
    for bid in reversed(ids[max(0, pos - 2):pos]):
        b = blocks_by_id[bid]
        if b["kind"] in ("paragraph", "list"):
            return b["text"][:200]
    return ""


def decide_roles(blocks: List[Dict[str, Any]], sections: List[Dict[str, Any]],
                 client: LLMClient, cfg: Dict[str, Any]) -> Tuple[Dict[str, Dict[str, Any]], List[Dict[str, str]]]:
    issues: List[Dict[str, str]] = []
    by_id = {b["id"]: b for b in blocks}
    sec_of: Dict[str, Dict[str, Any]] = {}
    for sec in sections:
        for tid in sec["table_block_ids"]:
            sec_of[tid] = sec
    roles: Dict[str, Dict[str, Any]] = {}
    for b in blocks:
        if b["kind"] != "table":
            continue
        tid = b["id"]
        sec = sec_of.get(tid)
        table = b["table"]
        if sec is None:
            roles[tid] = {"role": "ignore", "source": "rule", "reason": "table outside any section"}
            continue
```

```
        if table is None or not table["rows"]:
            reason = "table could not be decoded" if table is None else "table has no data rows"
            roles[tid] = {"role": "ignore", "source": "rule", "reason": reason}
            if sec["label"] in ("test_inputs", "expected_outputs", "other"):
                issues.append(make_issue("review", "roles", "table %s ignored: %s" % (tid, reason),
                                         tid, "check the table in the Word file"))
            continue
        label = sec["label"]
        caption = _caption(by_id, sec, tid)
        headers = [f["key"] for f in table["fields"]]
        kw = keyword_role(" ".join([sec["heading"], caption] + headers[:8]))
        default = {"test_inputs": "input", "expected_outputs": "expected"}.get(label)
        n_data_tables = sum(1 for x in sec["table_block_ids"]
                            if by_id[x]["table"] is not None and by_id[x]["table"]["rows"])
        ambiguous = (default is not None and (n_data_tables > 1 or (kw is not None and kw != default))) or \
                    (label == "other" and kw is None)
        if default is not None and not ambiguous:
            roles[tid] = {"role": default, "source": "section", "reason": "section label %s" % label}
            continue
        if default is None and label != "other":
            roles[tid] = {"role": "ignore", "source": "section", "reason": "section label %s" % label}
            continue
        if default is None and not ambiguous and kw is not None:
            roles[tid] = {"role": kw, "source": "rule", "reason": "keywords in heading/headers"}
            issues.append(make_issue("review", "roles", "table %s in an 'other' section was used as %s (keywords)" % (
    +tid, kw), tid))
            continue
        # ambiguous: ask the LLM, fall back to keywords/section
        first = ""
        if table["rows"]:
            r0 = table["rows"][0]["values"]
            first = "; ".join("%s=%s" % (k, ",".join("" if v is None else v for v in vals))
                              for k, vals in list(r0.items())[:6])
        role = None
        reason = ""
        if client.enabled:
            prompt = table_role_prompt(sec["heading"], caption, label, headers[:12], first,
                                       kw or default or "", table["n_rows"])
            try:
                ans = client.ask_json("table_role", SYSTEM_JSON, prompt, table_role_schema(ROLES))
                role, reason = ans["role"], ans["reason"]
                src = "llm"
            except LLMError as exc:
                issues.append(make_issue("review", "roles", "LLM could not decide the role of table %s (%s)" % (tid, e
    +xc), tid))
        if role is None:
            role, src = (kw or default or "ignore"), "rule"
            reason = "keyword/section fallback"
        roles[tid] = {"role": role, "source": src, "reason": reason}
        if src == "rule" and default is not None and kw is not None and kw != default:
            issues.append(make_issue("review", "roles", "table %s: heading suggests %s but section says %s; used %s" %
    + (tid, kw, default, role), tid))
        if role == "ignore" and label in ("test_inputs", "expected_outputs"):
            issues.append(make_issue("review", "roles", "table %s in a %s section was ignored" % (tid, label), tid))
    if not any(r["role"] in ("input", "expected", "mixed") for r in roles.values()):
        issues.append(make_issue("error", "roles", "no input or expected-output table was found; the test file will ha
    +ve no test cases",
                                 fix="check section headings, or set roles by hand in 02_roles.json and resume from st
    +age 3"))
    return roles, issues
```

testgen/recipes.py **(245 lines)**

```
"""Stage 3: one *recipe* per table says what every column means.

A recipe is the reviewable decision "column X of this table layout = variable Y,
and it is a test input / expected check / ...". Values never pass through the
LLM; code copies them from the decoded table using the recipe.

Recipe shape (JSON)
    {"layout_hash": "...", "role": "expected", "approved": false, "from_cache": false,
     "fields": {"Section A / Power Status": {"kind": "check", "variable": "power_status",
                "lane_meaning": "none", "source": "rule", "confidence": 1.0, "reason": ""}}}
kind: set_input | check | verify_only | ignore | env_func | env_computers | env_verify
source: config (table_column_map) | rule | llm | cache | default

How a field is resolved (first match wins)
    1. column empty everywhere                  -> ignore
    2. config table_column_map                  -> as configured
    3. header matches ignore_header_patterns    -> ignore (remarks, comments, ...)
    4. keyword/fuzzy match against symbols.csv  -> exact or clear fuzzy match
    5. ambiguous match                          -> LLM picks ONE candidate (or NONE)
    6. nothing close                            -> unresolved (kept, flagged, rendered as ??UNRESOLVED??)
Recipes are cached per (layout, role) in recipe_dir so the same layout never asks
the LLM twice. Edit the cached file and set "approved": true to lock it.
"""
from __future__ import annotations

```

```
import json
import os
import re
from typing import Any, Dict, List, Optional, Tuple

from .grounding import SymbolTable, normalize
from .issues import make_issue
from .llm_client import LLMClient, LLMError
from .prompts import SYSTEM_JSON, field_map_prompt
from .roles import EXPECTED_WORDS, INPUT_WORDS
from .schema import field_map_schema

KIND_HELP = {
    "set_input": "the value is SET as a test input",
    "check": "the value is CHECKED as an expected output",
    "verify_only": "the value is only looked at manually",
    "ignore": "not needed in the test file",
}
ROLE_KINDS = {"input": ["set_input", "verify_only", "ignore"],
              "expected": ["check", "verify_only", "ignore"],
              "mixed": ["set_input", "check", "verify_only", "ignore"]}
ALL_KINDS = ["set_input", "check", "verify_only", "ignore", "env_func", "env_computers", "env_verify"]
_STATUS_RANK = {"exact": 3, "fuzzy": 2, "ambiguous": 1, "none": 0}


def _texts(path: List[str]) -> List[str]:
    """Phrases to try for grounding: leaf, last two segments, whole path."""
    if not path:
        return []
    out = [path[-1]]
    if len(path) >= 2:
        out.append(" ".join(path[-2:]))
    if len(path) >= 3:
        out.append(" ".join(path))
    return out


def _lookup_column_map(cmap: Dict[str, Any], key: str, path: List[str]) -> Optional[Dict[str, Any]]:
    wanted = [key] + _texts(path)[::-1]
    index = {normalize(k): v for k, v in cmap.items()}
    for text in wanted:
        hit = index.get(normalize(text))
        if hit is not None:
            return {"variable": hit} if isinstance(hit, str) else dict(hit)
    return None


def _kind_from_keywords(text: str) -> Optional[str]:
    norm = " " + normalize(text) + " "
    n_in = sum(1 for w in INPUT_WORDS if w in norm)
    n_out = sum(1 for w in EXPECTED_WORDS if w in norm)
    if n_in > n_out:
        return "set_input"
    if n_out > n_in:
        return "check"
    return None


def _cache_path(recipe_dir: Optional[str], layout: str, role: str) -> Optional[str]:
    return os.path.join(recipe_dir, "%s_%s.json" % (layout, role)) if recipe_dir else None


def _load_cached(path: Optional[str], table: Dict[str, Any], symbols: SymbolTable) -> Optional[Dict[str, Any]]:
    if not path or not os.path.isfile(path):
        return None
    try:
        with open(path, "r", encoding="utf-8") as handle:
            rec = json.load(handle)
        fields = rec["fields"]
        keys = {f["key"] for f in table["fields"]}
        if set(fields) != keys:
            return None
        for entry in fields.values():
            if entry["kind"] not in ALL_KINDS:
                return None
            if entry.get("variable") is not None and not symbols.has_variable(entry["variable"]):
                return None
        return rec
    except (OSError, ValueError, KeyError, TypeError):
        return None                                  # unreadable cache: rebuild


def _save_cached(path: Optional[str], recipe: Dict[str, Any]) -> None:
    if not path:
        return
    try:
        os.makedirs(os.path.dirname(path), exist_ok=True)
        tmp = path + ".tmp"
        with open(tmp, "w", encoding="utf-8") as handle:
```

```
            json.dump(recipe, handle, indent=2, ensure_ascii=False)
        os.replace(tmp, path)
    except OSError:
        pass


def _lane_meaning(fld: Dict[str, Any], kind: str, cfg: Dict[str, Any], spec: Optional[Dict[str, Any]] = None) -> str:
    if fld["kind"] != "lanes":
        return "none"
    if spec and spec.get("lane_meaning") in ("computer", "channel", "bit", "none"):
        return spec["lane_meaning"]
    return cfg["lane_meaning"].get(kind, "none")


def _map_field(fld: Dict[str, Any], role: str, table: Dict[str, Any], tlabel: str,
               symbols: SymbolTable, client: LLMClient, cfg: Dict[str, Any],
               issues: List[Dict[str, str]]) -> Dict[str, Any]:
    key, path = fld["key"], fld["path"]
    loc = "%s / %s" % (tlabel, key)
    default_kind = {"input": "set_input", "expected": "check"}.get(role)
    values = [v for row in table["rows"] for v in row["values"][key] if v not in (None, "")]
    if not values:
        return {"kind": "ignore", "variable": None, "lane_meaning": "none", "source": "rule",
                "confidence": 1.0, "reason": "column has no values"}
    # 2. explicit configuration
    spec = _lookup_column_map(cfg["table_column_map"], key, path)
    if spec is not None:
        kind = spec.get("kind") or default_kind or _kind_from_keywords(key) or "check"
        var = spec.get("variable")
        if kind not in ALL_KINDS:
            issues.append(make_issue("review", "recipes", "table_column_map: unknown kind %r for %s" % (kind, key), lo
      +c))
            kind = default_kind or "check"
        if var is not None and not symbols.has_variable(var):
            issues.append(make_issue("review", "recipes", "table_column_map: variable %r is not in symbols.csv" % var,
      + loc))
            var = None
        return {"kind": kind, "variable": var, "lane_meaning": _lane_meaning(fld, kind, cfg, spec),
                "source": "config", "confidence": 1.0, "reason": "table_column_map"}
    # 3. headers that never carry test data
    leaf = path[-1] if path else key
    for pat in cfg["ignore_header_patterns"]:
        if re.search(pat, leaf.strip(), re.I):
            return {"kind": "ignore", "variable": None, "lane_meaning": "none", "source": "rule",
                    "confidence": 1.0, "reason": "header matches ignore pattern %r" % pat}
    # 4. rule-based grounding over several phrases, keep the best result
    best: Optional[Dict[str, Any]] = None
    pool: Dict[str, float] = {}
    for text in _texts(path) or [key]:
        res = symbols.resolve(text, cfg["grounding"])
        for var, sc in res["candidates"]:
            pool[var] = max(pool.get(var, 0.0), sc)
        rank = (_STATUS_RANK[res["status"]], res["score"])
        if best is None or rank > (_STATUS_RANK[best["status"]], best["score"]):
            best = res
    assert best is not None
    kind = default_kind or _kind_from_keywords(" ".join(path)) or None
    if best["status"] in ("exact", "fuzzy") and kind:
        return {"kind": kind, "variable": best["variable"], "lane_meaning": _lane_meaning(fld, kind, cfg),
                "source": "rule", "confidence": best["score"], "reason": best["status"] + " match"}
    # 5. ambiguous (or mixed table): let the LLM choose among code-computed candidates
    cands = sorted(pool.items(), key=lambda kv: (-kv[1], kv[0]))[:int(cfg["grounding"]["top_n"])]
    cands = [c for c in cands if c[1] >= float(cfg["grounding"]["floor"])]
    if best["status"] in ("exact", "fuzzy"):
        cands = [(best["variable"], best["score"])]
    if cands and client.enabled:
        kinds = ROLE_KINDS.get(role, ROLE_KINDS["mixed"])
        samples = []
        for row in table["rows"][:3]:
            vals = ",".join("" if v is None else v for v in row["values"][key])
            samples.append(vals)
        prompt = field_map_prompt(role, key, fld["kind"], fld["lane_labels"], samples,
                                  [c[0] for c in cands], {k: KIND_HELP[k] for k in kinds})
        try:
            ans = client.ask_json("field_map", SYSTEM_JSON, prompt,
                                  field_map_schema([c[0] for c in cands], kinds))
            var = None if ans["variable"] == "NONE" else ans["variable"]
            k = ans["kind"] if var is not None or ans["kind"] == "ignore" else "ignore"
            if var is None and k != "ignore":
                k = "ignore"
            return {"kind": k, "variable": var, "lane_meaning": _lane_meaning(fld, k, cfg),
                    "source": "llm", "confidence": 0.5, "reason": ans["reason"]}
        except LLMError as exc:
            issues.append(make_issue("review", "recipes", "LLM could not map column %s (%s)" % (key, exc), loc))
    # 6. unresolved: keep the column so nothing disappears silently
    final_kind = kind or default_kind or "check"
    var = best["variable"] if best["status"] in ("exact", "fuzzy") else None
    issues.append(make_issue(
        "review", "recipes",
```

```
        "column %r could not be linked to a variable (best candidates: %s)"
        % (key, ", ".join(c[0] for c in cands[:3]) or "none"), loc,
        "add it to table_column_map in config.json or edit the recipe file and set approved=true"))
    return {"kind": final_kind, "variable": var, "lane_meaning": _lane_meaning(fld, final_kind, cfg),
            "source": "default", "confidence": 0.0, "reason": "unresolved"}


def build_recipes(blocks: List[Dict[str, Any]], roles: Dict[str, Dict[str, Any]], symbols: SymbolTable,
                  client: LLMClient, cfg: Dict[str, Any], recipe_dir: Optional[str]
                  ) -> Tuple[Dict[str, Dict[str, Any]], List[Dict[str, str]]]:
    """Return ({table block id: recipe}, issues)."""
    issues: List[Dict[str, str]] = []
    recipes: Dict[str, Dict[str, Any]] = {}
    for b in blocks:
        if b["kind"] != "table" or b["table"] is None:
            continue
        role = roles.get(b["id"], {}).get("role", "ignore")
        if role == "ignore":
            continue
        table = b["table"]
        tlabel = "T%d" % (b["table_index"] + 1)
        path = _cache_path(recipe_dir, table["layout_hash"], role)
        cached = _load_cached(path, table, symbols)
        if cached is not None:
            cached["from_cache"] = True
            recipes[b["id"]] = cached
            if not cached.get("approved"):
                issues.append(make_issue("review", "recipes",
                                         "recipe for %s (layout %s) was reused from cache but is not approved"
                                         % (tlabel, table["layout_hash"]), tlabel,
                                         "check %s and set \"approved\": true" % path))
            continue
        fields: Dict[str, Dict[str, Any]] = {}
        for fld in table["fields"]:
            fields[fld["key"]] = _map_field(fld, role, table, tlabel, symbols, client, cfg, issues)
        recipe = {"layout_hash": table["layout_hash"], "role": role, "approved": False,
                  "from_cache": False, "table_block": b["id"], "fields": fields}
        recipes[b["id"]] = recipe
        _save_cached(path, recipe)
        issues.append(make_issue("review", "recipes",
                                 "recipe for %s (layout %s) was generated automatically and is not approved"
                                 % (tlabel, table["layout_hash"]), tlabel,
                                 "check %s and set \"approved\": true" % (path or "03_recipes.json")))
    return recipes, issues
```

`testgen/assemble.py` **(210 lines)**

```
"""Stage 4: build interleaved test cases from the decoded tables and recipes.

Input and expected-output tables stay separate in the Word file; here they are
joined into one record per test case so every later stage (render, validate,
review) works on one case at a time.

Join rule
    * tables with a usable id column (unique ids) are joined by id
    * otherwise by row position
    * row-count / id mismatches are reported; unmatched rows are never dropped
    * several tables with the same role and layout are concatenated; tables with
      different layouts of the same role are joined to the same cases by position

Case shape
    {"id": "1", "ordinal": 1,
     "setup_test_case_environment": {"func": [], "target_computer": []},
     "verify_test_case_environment": [<variable text>],
     "steps": [{"source": {"input_rows": [...], "expected_rows": [...]},
                "set_input": [item], "check_expected_output": [item], "verify_only": [item]}]}
    item = {"variable": str|None, "label": column key, "value": str|None,
            "lanes": [str|None]|None, "lane_labels": [...], "lane_meaning": ..., "source": "T3 row 2 / ..."}
Environment columns (kinds env_func / env_computers / env_verify) exist only when
configured through table_column_map; otherwise the environment stays empty and
ONE review issue is raised (or case_environment_default from the config is used).
"""
from __future__ import annotations

import copy
import re
from typing import Any, Dict, List, Optional, Tuple

from .issues import make_issue


def _idkey(text: str) -> str:
    t = str(text).strip().lower()
    return str(int(t)) if t.isdigit() else t


def _make_item(entry: Dict[str, Any], fld: Dict[str, Any], vals: List[Optional[str]], source: str
               ) -> Optional[Dict[str, Any]]:
```

```
    if fld["kind"] == "lanes":
        if all(v is None for v in vals):
            return None
        return {"variable": entry.get("variable"), "label": fld["key"], "value": None,
                "lanes": list(vals), "lane_labels": list(fld["lane_labels"]),
                "lane_meaning": entry.get("lane_meaning", "none"), "source": source}
    if vals[0] is None:
        return None
    return {"variable": entry.get("variable"), "label": fld["key"], "value": vals[0], "lanes": None,
            "lane_labels": [], "lane_meaning": "none", "source": source}


def _group_parts(blocks: List[Dict[str, Any]], roles: Dict[str, Dict[str, Any]],
                 recipes: Dict[str, Dict[str, Any]], issues: List[Dict[str, str]]) -> List[Dict[str, Any]]:
    parts: List[Dict[str, Any]] = []
    for b in blocks:
        if b["kind"] != "table" or b["table"] is None:
            continue
        role = roles.get(b["id"], {}).get("role", "ignore")
        recipe = recipes.get(b["id"])
        if role == "ignore" or recipe is None:
            continue
        layout = b["table"]["layout_hash"]
        tlabel = "T%d" % (b["table_index"] + 1)
        part = next((p for p in parts if p["role"] == role and p["layout"] == layout), None)
        if part is None:
            part = {"role": role, "layout": layout, "table": b["table"], "recipe": recipe, "rows": [],
                    "tables": []}
            parts.append(part)
        else:
            issues.append(make_issue("review", "assemble",
                                     "%s has the same layout as an earlier %s table; rows are appended as further test
      + cases"
                                     % (tlabel, role), tlabel))
        part["tables"].append(tlabel)
        for row in b["table"]["rows"]:
            part["rows"].append({"row": row, "tlabel": tlabel})
    for role in ("input", "expected", "mixed"):
        n = len([p for p in parts if p["role"] == role])
        if n > 1:
            issues.append(make_issue("review", "assemble",
                                     "%d %s tables with different layouts are joined to the same cases by row position
      +" % (n, role)))
    return parts


def _split_funcs(text: str) -> List[str]:
    """'in_water, boot' -> ['in_water', 'boot']"""
    return [t.strip() for t in re.split(r"[,;\n/]+", text) if t.strip()]


def assemble_cases(blocks: List[Dict[str, Any]], roles: Dict[str, Dict[str, Any]],
                   recipes: Dict[str, Dict[str, Any]], cfg: Dict[str, Any]
                   ) -> Tuple[List[Dict[str, Any]], List[Dict[str, str]]]:
    issues: List[Dict[str, str]] = []
    parts = _group_parts(blocks, roles, recipes, issues)
    if not parts:
        return [], issues

    # --- decide the order and keys of the cases -------------------------------
    id_part_maps: List[Tuple[Dict[str, Dict[str, Any]], Dict[str, str]]] = []
    for p in parts:
        ids = [_idkey(r["row"]["row_id"]) for r in p["rows"]]
        usable = p["table"]["id_column"] and len(set(ids)) == len(ids) and all(ids)
        p["by_id"] = {k: r for k, r in zip(ids, p["rows"])} if usable else None
        p["ids"] = ids if usable else None
    order: List[str] = []
    display: Dict[str, str] = {}
    id_parts = [p for p in parts if p["by_id"] is not None]
    for p in id_parts:
        for k, r in zip(p["ids"], p["rows"]):
            if k not in display:
                order.append(k)
                display[k] = r["row"]["row_id"]
    if len(id_parts) > 1:
        sets = [set(p["ids"]) for p in id_parts]
        if any(s != sets[0] for s in sets[1:]):
            only = sorted(set().union(*sets) - set.intersection(*sets))
            issues.append(make_issue("review", "assemble",
                                     "test ids differ between tables (not in all tables: %s)" % ", ".join(only[:10]),
                                     fix="check the id columns of the input and expected-output tables"))
    positional = [p for p in parts if p["by_id"] is None]
    n_pos = max([len(p["rows"]) for p in positional] or [0])
    if not id_parts:
        order = ["#%d" % (i + 1) for i in range(n_pos)]
        display = {k: str(i + 1) for i, k in enumerate(order)}
    else:
        for i in range(len(order), n_pos):                  # positional part longer than the id order
            key = "#%d" % (i + 1)
```

```
            order.append(key)
            display[key] = str(i + 1)
    counts = {"%s %s" % (p["role"], "+".join(p["tables"])): len(p["rows"]) for p in parts}
    if len(set(len(p["rows"]) for p in parts)) > 1:
        issues.append(make_issue("review", "assemble",
                                 "row counts differ between tables (%s); unmatched rows are kept in their own cases"
                                 % ", ".join("%s: %d" % kv for kv in counts.items()),
                                 fix="check that every test case has an input row and an expected-output row"))

    # --- build the cases -------------------------------------------------------
    env_default = cfg.get("case_environment_default")
    env_seen = False
    cases: List[Dict[str, Any]] = []
    for n, key in enumerate(order, start=1):
        env = copy.deepcopy(env_default) if isinstance(env_default, dict) else {}
        case = {"id": display[key], "ordinal": n,
                "setup_test_case_environment": {"func": list(env.get("func", [])),
                                                "target_computer": list(env.get("target_computer", []))},
                "verify_test_case_environment": list(env.get("verify", [])),
                "steps": []}
        step = {"source": {"input_rows": [], "expected_rows": []},
                "set_input": [], "check_expected_output": [], "verify_only": []}
        for p in parts:
            if p["by_id"] is not None:
                ref = p["by_id"].get(key)
            else:
                idx = int(key[1:]) - 1 if key.startswith("#") else n - 1
                ref = p["rows"][idx] if idx < len(p["rows"]) else None
            if ref is None:
                continue
            row, tlabel = ref["row"], ref["tlabel"]
            bucket = "input_rows" if p["role"] == "input" else "expected_rows"
            step["source"][bucket].append("%s row %s" % (tlabel, row["row_id"]))
            for fld in p["table"]["fields"]:
                entry = p["recipe"]["fields"].get(fld["key"])
                if not entry or entry["kind"] == "ignore":
                    continue
                vals = row["values"][fld["key"]]
                src = "%s row %s / %s" % (tlabel, row["row_id"], fld["key"])
                kind = entry["kind"]
                if kind in ("set_input", "check", "verify_only"):
                    item = _make_item(entry, fld, vals, src)
                    if item is None:
                        continue
                    if kind == "set_input":
                        step["set_input"].append(item)
                    elif kind == "check":
                        step["check_expected_output"].append(item)
                    else:
                        step["verify_only"].append(item)
                elif kind in ("env_func", "env_computers", "env_verify"):
                    env_seen = True
                    text = ",".join(v for v in vals if v)
                    if not text:
                        continue
                    setup = case["setup_test_case_environment"]
                    if kind == "env_func":
                        for name in _split_funcs(text):
                            known = cfg.get("funcs") or []
                            if known and name not in known:
                                issues.append(make_issue("review", "assemble", "func %r is not in the configured funct
  +ion list" % name, src))
                            if name not in setup["func"]:
                                setup["func"].append(name)
                    elif kind == "env_computers":
                        bits = [v for v in vals if v is not None] if fld["kind"] == "lanes" else list(text)
                        if bits and all(str(b) in ("0", "1") for b in bits):
                            setup["target_computer"] = [int(b) for b in bits]
                        else:
                            issues.append(make_issue("review", "assemble", "computer selection %r is not a list of 0/1
  +" % text, src))
                    else:
                        for name in _split_funcs(text):
                            if name not in case["verify_test_case_environment"]:
                                case["verify_test_case_environment"].append(name)
        case["steps"].append(step)
        if not step["set_input"] and not step["check_expected_output"] and not step["verify_only"]:
            issues.append(make_issue("review", "assemble", "test case %s has no inputs and no expected outputs" % case
  +["id"], "case %s" % case["id"]))
        cases.append(case)
    if cases and not env_seen and not isinstance(env_default, dict):
        issues.append(make_issue("review", "assemble",
                                 "no column describes the test-case environment (func and computers); it was left empt
  +y for all %d cases" % len(cases),
                                 fix="map a column with kind env_func / env_computers in table_column_map, or set case
  +_environment_default in config.json"))
    return cases, issues
```

testgen/notes.py **(105 lines)**

```
"""Stage 5: facts from the paragraphs (initialization hints, constraints).

Test cases come from tables. Paragraphs only contribute
    - hints about the global initialization (functions, checks, manual verifies)
    - constraints / notes
The initialization itself is handled by another file, so by default these are
written to review.md, and only rendered when config emit_global_init is true.

Two LLM passes per text window: (A) rewrite as short lines under fixed headings,
(B) grammar-constrained JSON. Every extracted item must carry `evidence`, a
quote that really occurs in the source; items whose evidence is not found are
dropped and reported (possible hallucination).
"""
from __future__ import annotations

import re
from typing import Any, Dict, List, Tuple

from .grounding import normalize
from .issues import make_issue
from .llm_client import LLMClient, LLMError
from .prompts import NOTES_SYSTEM, notes_extract_prompt, notes_normalize_prompt
from .schema import notes_schema

WINDOW_CHARS = 3000


def _windows(texts: List[str], limit: int = WINDOW_CHARS) -> List[str]:
    """Group lines into windows below `limit` characters (never split a line)."""
    out, cur = [], ""
    for t in texts:
        if cur and len(cur) + len(t) + 1 > limit:
            out.append(cur)
            cur = ""
        cur = (cur + "\n" + t) if cur else t
    if cur:
        out.append(cur)
    return out


def _in_source(evidence: str, source: str) -> bool:
    e, s = normalize(evidence), normalize(source)
    return bool(e) and e in s


def extract_notes(blocks: List[Dict[str, Any]], sections: List[Dict[str, Any]], client: LLMClient,
                  cfg: Dict[str, Any]) -> Tuple[Dict[str, Any], List[Dict[str, str]]]:
    issues: List[Dict[str, str]] = []
    notes: Dict[str, Any] = {"init": {"funcs": [], "test_variables": [], "verify_variables": [], "source_sections": []
      +},
                             "constraints": [], "raw": {"initialization": [], "constraints": []}}
    by_id = {b["id"]: b for b in blocks}
    for label in ("initialization", "constraints"):
        lines: List[str] = []
        for sec in sections:
            if sec["label"] != label:
                continue
            if label == "initialization":
                notes["init"]["source_sections"].append(sec["id"])
            for bid in sec["block_ids"]:
                b = by_id[bid]
                if b["text"]:
                    lines.append(b["text"])
        notes["raw"][label] = lines
        if not lines:
            continue
        if not client.enabled:
            issues.append(make_issue("review", "notes", "%s text was not analysed (no LLM); read it in review.md" % la
       +bel))
            if label == "constraints":
                notes["constraints"] = [{"text": t, "evidence": t} for t in lines]
            continue
        for window in _windows(lines):
            try:
                normalized = client.ask_text("notes_normalize", NOTES_SYSTEM,
                                             notes_normalize_prompt(window, cfg["funcs"]))
                ans = client.ask_json("notes_extract", NOTES_SYSTEM, notes_extract_prompt(window, normalized),
                                      notes_schema(cfg["funcs"]))
            except LLMError as exc:
                issues.append(make_issue("review", "notes", "could not analyse the %s text (%s); read it in review.md"
        + % (label, exc)))
                if label == "constraints":
                    notes["constraints"].extend({"text": t, "evidence": t} for t in window.split("\n"))
                continue
            dropped = 0
            for f in ans["funcs"]:
```

```
                if f not in notes["init"]["funcs"]:
                    notes["init"]["funcs"].append(f)
            for item in ans["test_variables"]:
                if _in_source(item["evidence"], window):
                    notes["init"]["test_variables"].append(item)
                else:
                    dropped += 1
            for item in ans["verify_variables"]:
                if _in_source(item["evidence"], window):
                    notes["init"]["verify_variables"].append(item)
                else:
                    dropped += 1
            for item in ans["constraints"]:
                if _in_source(item["evidence"], window):
                    notes["constraints"].append(item)
                else:
                    dropped += 1
            if dropped:
                issues.append(make_issue("review", "notes",
                                         "%d extracted item(s) dropped: their evidence quote is not in the source text
      +" % dropped,
                                         label, "read the %s text in review.md" % label))
    return notes, issues
```

`testgen/render.py` **(203 lines)**

```
"""Stage 8: spec JSON -> tpf text. No LLM, no I/O.

The renderer only trusts the spec: every item already carries its `symbol`
(or None when unresolved). Syntax lives in config['formats'] (templates), so a
syntax change never touches code.

Rules
    scalar input     <sym>[<default input mask>] <-> value
    scalar check     <sym>[<default check mask>] <-> value/
    lane check       computers that share a value go on ONE line:
                     [1,0,1,0] -> sym[1010] <-> 1/  and  sym[0101] <-> 0/
    channel lanes    cannot be written until the syntax is known -> '# TODO' line
                     (or masks when input_lane_render = "mask")
    unresolved item  ??UNRESOLVED: <label> = <values>??  (never skipped silently)
"""
from __future__ import annotations

import re
from typing import Any, Dict, List, Optional, Tuple

from .config import DEFAULT_FORMATS
from .issues import make_issue


def clean(value: Any) -> str:
    """One-line, trimmed text."""
    return re.sub(r"\s+", " ", str(value)).strip()


def lane_groups(values: List[Optional[str]]) -> List[Tuple[str, str]]:
    """[(mask, value)] with one entry per distinct value, in first-seen order.
    Lanes whose value is None ("don't care") are left out of every mask."""
    order: List[str] = []
    members: Dict[str, List[int]] = {}
    for i, raw in enumerate(values):
        if raw is None:
            continue
        v = clean(raw)
        if v == "":
            continue
        if v not in members:
            members[v] = []
            order.append(v)
        members[v].append(i)
    out = []
    for v in order:
        idx = set(members[v])
        out.append(("".join("1" if i in idx else "0" for i in range(len(values))), v))
    return out


class Renderer:
    def __init__(self, cfg: Dict[str, Any]) -> None:
        self.cfg = cfg
        self.issues: List[Dict[str, str]] = []
        self._agg: Dict[Tuple[str, str], Dict[str, Any]] = {}   # repeated findings, one issue per column
        self.lines: List[str] = []
        self.stats = {"unresolved": 0, "todo": 0, "items": 0}

    def fmt(self, _key: str, **kw: Any) -> str:
        """Apply format template `_key` (its placeholders, e.g. name=, arrive as keywords)."""
        template = self.cfg["formats"].get(_key) or DEFAULT_FORMATS[_key]
```

```
        try:
            return template.format(**kw)
        except (KeyError, IndexError, ValueError):
            self.issues.append(make_issue("error", "render", "format %r could not be applied; default used" % _key))
            return DEFAULT_FORMATS[_key].format(**kw)

    def emit(self, line: str) -> None:
        self.lines.append(line)

    def note(self, kind: str, key: str, message: str, location: str, fix: str) -> None:
        """Remember a finding that repeats for every row; reported once with a count."""
        entry = self._agg.setdefault((kind, key), {"n": 0, "message": message, "location": location, "fix": fix})
        entry["n"] += 1

    def flush_notes(self) -> None:
        for entry in self._agg.values():
            more = "" if entry["n"] == 1 else " (%d occurrences, first shown)" % entry["n"]
            self.issues.append(make_issue("review", "render", entry["message"] + more, entry["location"], entry["fix"]
      +))
        self._agg.clear()

    def unresolved(self, item: Dict[str, Any], loc: str) -> None:
        vals = item.get("lanes") if item.get("lanes") is not None else [item.get("value")]
        shown = ",".join("" if v is None else clean(v) for v in vals)
        label = item.get("label") or item.get("variable") or "?"
        self.emit(self.fmt("unresolved", text="%s = %s" % (label, shown)))
        self.stats["unresolved"] += 1
        self.note("unresolved", label, "no symbol for %r; line written as ??UNRESOLVED??" % label,
                  item.get("source") or loc, "add the variable to symbols.csv or fix the recipe")

    def item(self, item: Dict[str, Any], kind: str, loc: str) -> None:
        """kind: set_input | check | verify"""
        self.stats["items"] += 1
        sym = item.get("symbol")
        if not sym:
            self.unresolved(item, loc)
            return
        if kind == "verify":
            self.emit(self.fmt("verify", sym=sym))
            return
        mask_default = self.cfg["default_masks"]["input" if kind == "set_input" else "check"]
        lanes = item.get("lanes")
        if lanes is None:
            value = clean(item.get("value", ""))
            if value:
                self.emit(self.fmt(kind, sym=sym, mask=mask_default, value=value))
            return
        meaning = item.get("lane_meaning", "none")
        as_mask = meaning == "computer" or (meaning == "channel" and self.cfg["input_lane_render"] == "mask")
        if as_mask:
            if meaning == "computer" and len(lanes) != self.cfg["num_computers"]:
                self.note("masklen", sym, "%s has %d lane values but num_computers is %d"
                          % (sym, len(lanes), self.cfg["num_computers"]), item.get("source") or loc, "")
            for mask, value in lane_groups(lanes):
                self.emit(self.fmt(kind, sym=sym, mask=mask, value=value))
            return
        shown = ",".join("-" if v is None else clean(v) for v in lanes)
        self.emit(self.fmt("todo", text="%s: %d %s lanes [%s]; syntax not defined" % (sym, len(lanes), meaning, shown)
      +))
        self.stats["todo"] += 1
        self.note("todo", sym, "%s has %s lanes; no tpf syntax is defined for them (TODO line written)" % (sym, meanin
      +g),
                  item.get("source") or loc, "show the target syntax, or set input_lane_render to 'mask'")

    def global_init(self, init: Dict[str, Any]) -> None:
        funcs = init.get("funcs", [])
        tests = init.get("test_variables", [])
        verifies = init.get("verify_variables", [])
        if not (funcs or tests or verifies):
            return
        self.emit(self.fmt("section_init"))
        for name in funcs:
            self.emit(self.fmt("init_func", name=name))
        for t in tests:
            if t.get("symbol"):
                self.emit(self.fmt("init_check", sym=t["symbol"], value=clean(t.get("expected_value", ""))))
            else:
                self.unresolved({"label": t.get("variable_text", "?"), "value": t.get("expected_value")}, "global init
      +")
        for v in verifies:
            if v.get("symbol"):
                self.emit(self.fmt("init_verify", sym=v["symbol"]))
            else:
                self.unresolved({"label": v.get("variable_text", "?"), "value": ""}, "global init")

    def case(self, case: Dict[str, Any]) -> None:
        loc = "case %s" % case["id"]
        self.emit(self.fmt("case_header", n=case["ordinal"], id=case["id"]))
        setup = case["setup_test_case_environment"]
```

```
        if setup.get("func") or setup.get("target_computer"):
            self.emit(self.fmt("section_env"))
            for name in setup.get("func", []):
                self.emit(self.fmt("env_func", name=name))
            comp = setup.get("target_computer", [])
            if comp:
                if all(str(c) in ("0", "1") for c in comp):
                    self.emit(self.fmt("env_computers", mask="".join(str(int(c)) for c in comp)))
                else:
                    self.issues.append(make_issue("error", "render", "target_computer %r is not a list of 0/1" % (comp
        +,), loc))
        verify = case.get("verify_test_case_environment", [])
        if verify:
            self.emit(self.fmt("section_env_verify"))
            exp = self.cfg.get("env_expectations", {})
            for v in verify:
                var = v.get("variable") if isinstance(v, dict) else None
                sym = v.get("symbol") if isinstance(v, dict) else None
                label = v.get("variable_text", var) if isinstance(v, dict) else str(v)
                if not sym:
                    self.unresolved({"label": label, "value": ""}, loc)
                    continue
                value = None
                for func in setup.get("func", []):
                    if var in exp.get(func, {}):
                        value = exp[func][var]
                if value is None:
                    self.emit(self.fmt("env_verify", sym=sym))
                else:
                    self.emit(self.fmt("env_check", sym=sym, value=clean(value)))
        steps = case.get("steps", [])
        for s_idx, step in enumerate(steps, start=1):
            if len(steps) > 1:
                self.emit(self.fmt("step_header", n=s_idx))
            if step.get("set_input"):
                self.emit(self.fmt("section_inputs"))
                for it in step["set_input"]:
                    self.item(it, "set_input", loc)
            if step.get("check_expected_output") or step.get("verify_only"):
                self.emit(self.fmt("section_checks"))
                for it in step.get("check_expected_output", []):
                    self.item(it, "check", loc)
                for it in step.get("verify_only", []):
                    self.item(it, "verify", loc)


def render_spec(spec: Dict[str, Any], cfg: Dict[str, Any]) -> Tuple[List[str], List[Dict[str, str]], Dict[str, int]]:
    """Return (lines, issues, stats)."""
    r = Renderer(cfg)
    if cfg.get("emit_global_init"):
        r.global_init(spec.get("global_init", {}))
    for case in spec.get("test_cases", []):
        r.case(case)
    if not spec.get("test_cases"):
        r.emit("# no test cases were produced (see review.md)")
    r.flush_notes()
    return r.lines, r.issues, dict(r.stats, lines=len(r.lines))
```

`testgen/validate.py` **(67 lines)**

```
"""Stage 7: whole-spec validation. Findings become issues; nothing is raised.

Checks
    unknown variables / missing symbols, unresolved items
    mask lengths (lane count versus num_computers for computer lanes)
    conflicting values for the same symbol+mask inside one step
    empty cases, empty spec
    coverage: useful sections or tables that nothing used
"""
from __future__ import annotations

from typing import Any, Dict, List, Set, Tuple

from .issues import make_issue
from .render import lane_groups


def validate_spec(spec: Dict[str, Any], sections: List[Dict[str, Any]], roles: Dict[str, Dict[str, Any]],
                  recipes: Dict[str, Dict[str, Any]], cfg: Dict[str, Any]) -> List[Dict[str, str]]:
    issues: List[Dict[str, str]] = []
    cases = spec.get("test_cases", [])
    if not cases:
        issues.append(make_issue("error", "validate", "the spec contains no test cases"))
    n_unres = 0
    for case in cases:
        loc = "case %s" % case["id"]
        for s_idx, step in enumerate(case.get("steps", []), start=1):
            for bucket, mask_key in (("set_input", "input"), ("check_expected_output", "check")):
```

```
                seen: Dict[Tuple[str, str], str] = {}
                for item in step.get(bucket, []):
                    if not item.get("symbol"):
                        n_unres += 1
                        continue
                    lanes = item.get("lanes")
                    if lanes is None:
                        pairs = [(cfg["default_masks"][mask_key], str(item.get("value")))]
                    elif item.get("lane_meaning") == "computer":
                        if len(lanes) != cfg["num_computers"]:
                            issues.append(make_issue("review", "validate",
                                                     "%s: %d lanes, expected %d computers" % (item["symbol"], len(lane
      +s), cfg["num_computers"]),
                                                     item.get("source", loc)))
                        pairs = lane_groups(lanes)
                    else:
                        continue
                    for mask, value in pairs:
                        # a symbol written twice for overlapping computers with different values
                        for (sym, other_mask), other_val in list(seen.items()):
                            if sym == item["symbol"] and other_val != value and any(
                                    a == "1" and b == "1" for a, b in zip(mask, other_mask)):
                                issues.append(make_issue(
                                    "review", "validate",
                                    "%s is given two different values (%s, %s) for overlapping computers in %s step %d
      +"
                                    % (item["symbol"], other_val, value, bucket, s_idx), item.get("source", loc)))
                        seen[(item["symbol"], mask)] = value
    if n_unres:
        issues.append(make_issue("review", "validate", "%d item(s) have no symbol (rendered as ??UNRESOLVED??)" % n_un
      +res))
    used_tables = {tid for tid, r in recipes.items()}
    for sec in sections:
        if sec["label"] in ("test_inputs", "expected_outputs"):
            if not sec["table_block_ids"] and sec["block_ids"]:
                issues.append(make_issue("review", "validate",
                                         "section %r (%s) has no table; its text was not turned into test steps" % (se
      +c["heading"], sec["label"]),
                                         sec["id"], "describe the steps in a table, or add support for prose steps"))
            for tid in sec["table_block_ids"]:
                if tid not in used_tables and roles.get(tid, {}).get("role") != "ignore":
                    issues.append(make_issue("review", "validate", "table %s in %r was not used" % (tid, sec["heading"
      +]), tid))
    return issues
```

testgen/state.py **(45 lines)**

```
# NOTE: no "from __future__ import annotations" here on purpose. LangGraph reads the
# Annotated[...] reducers from the real type hints of RunState at graph-build time.
import operator
from typing import Any, Dict, List, Optional, TypedDict

try:                                    # Python 3.9+
    from typing import Annotated
except ImportError:                     # Python 3.8
    from typing_extensions import Annotated  # type: ignore


class RunState(TypedDict, total=False):
    """Shared state of the graph. Everything in it is JSON-serialisable.

    Written by stage        keys
        (start)             input_path
        load                blocks
        sections            sections
        roles               roles
        recipes             recipes
        assemble            cases
        notes               notes
        ground              spec
        validate            validation
        render              outputs
        every stage         issues, stage_log (lists that are APPENDED, see reducers)
        any stage           fatal (a message; stops the run)
    """
    input_path: str
    blocks: List[Dict[str, Any]]
    sections: List[Dict[str, Any]]
    roles: Dict[str, Dict[str, Any]]
    recipes: Dict[str, Dict[str, Any]]
    cases: List[Dict[str, Any]]
    notes: Dict[str, Any]
    spec: Dict[str, Any]
    validation: List[Dict[str, Any]]
    outputs: Dict[str, Any]
    fatal: Optional[str]
    issues: Annotated[List[Dict[str, Any]], operator.add]
    stage_log: Annotated[List[Dict[str, Any]], operator.add]
```

```

# Same reducers for the built-in sequential engine, so both engines behave alike.
REDUCERS = {"issues": operator.add, "stage_log": operator.add}
```

testgen/stages.py **(303 lines)**

```
"""The nine pipeline stages and the wrapper that makes each one safe.

A stage function has the signature   fn(ctx, state) -> (update_dict, issues)
and never talks to LangGraph. `make_node` wraps it so that
    * an exception becomes an issue plus a rule-based fallback result
      (strict mode and the load stage turn it into a fatal error instead)
    * a FatalError stops the run cleanly
    * the result is written to work/NN_<stage>.json (checkpoint)
    * with --resume-from N, stages before N are read from their checkpoints
    * the node always returns a stage_log entry (LangGraph requires a state update)
"""
from __future__ import annotations

import copy
import json
import os
import time
import traceback
from typing import Any, Callable, Dict, List, Optional, Tuple

from .assemble import assemble_cases
from .docx_loader import load_blocks
from .grounding import SymbolTable, choose_with_llm
from .issues import FatalError, dedupe, make_issue
from .llm_client import LLMClient, LLMError
from .notes import extract_notes
from .recipes import build_recipes
from .render import render_spec
from .roles import decide_roles
from .sections import build_sections, label_sections
from .validate import validate_spec


class RunContext:
    """Everything the stages need besides the state: config, symbols, LLM, paths."""

    def __init__(self, cfg: Dict[str, Any], symbols: SymbolTable, client: LLMClient, input_path: str,
                 out_dir: str, recipe_dir: Optional[str] = None, strict: bool = False,
                 resume_from: Optional[int] = None, stop_after: Optional[int] = None) -> None:
        self.cfg = cfg
        self.symbols = symbols
        self.client = client
        self.input_path = input_path
        self.out_dir = out_dir
        self.work_dir = os.path.join(out_dir, "work")
        self.recipe_dir = recipe_dir
        self.strict = strict or bool(cfg.get("strict"))
        self.resume_from = resume_from
        self.stop_after = stop_after
        self.stem = os.path.splitext(os.path.basename(input_path))[0] or "spec"

    def path(self, suffix: str) -> str:
        return os.path.join(self.out_dir, self.stem + suffix)


# ---------------------------------------------------------------------------
# helpers
# ---------------------------------------------------------------------------
def write_json(path: str, data: Any) -> None:
    os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
    tmp = path + ".tmp"
    with open(tmp, "w", encoding="utf-8") as handle:
        json.dump(data, handle, indent=2, ensure_ascii=False, default=str)
    os.replace(tmp, path)


def _ground_name(ctx: RunContext, text: str, context: str, issues: List[Dict[str, str]], loc: str
                 ) -> Optional[str]:
    """Free text -> canonical variable (rules first, LLM choice among candidates second)."""
    res = ctx.symbols.resolve(text, ctx.cfg["grounding"])
    if res["status"] in ("exact", "fuzzy"):
        return res["variable"]
    if res["status"] == "ambiguous" and ctx.client.enabled:
        try:
            return choose_with_llm(ctx.client, ctx.symbols, text, res["candidates"], context)
        except LLMError as exc:
            issues.append(make_issue("review", "ground", "LLM could not choose a variable for %r (%s)" % (text, exc),
                                     loc))
    issues.append(make_issue("review", "ground", "no variable found for %r" % text, loc,
                             "add it to symbols.csv (variable or aliases)"))
    return None
```

```
 81
 82
 83  # ---------------------------------------------------------------------------
 84  # stage functions
 85  # ---------------------------------------------------------------------------
 86  def stage_load(ctx: RunContext, state: Dict[str, Any]) -> Tuple[Dict[str, Any], List[Dict[str, str]]]:
 87      blocks, issues = load_blocks(ctx.input_path, ctx.cfg)
 88      return {"blocks": blocks}, issues
 89
 90
 91  def stage_sections(ctx: RunContext, state: Dict[str, Any]) -> Tuple[Dict[str, Any], List[Dict[str, str]]]:
 92      sections = build_sections(state.get("blocks", []), int(ctx.cfg["sections"]["preview_chars"]))
 93      issues = label_sections(sections, ctx.client, ctx.cfg)
 94      return {"sections": sections}, issues
 95
 96
 97  def stage_roles(ctx: RunContext, state: Dict[str, Any]) -> Tuple[Dict[str, Any], List[Dict[str, str]]]:
 98      roles, issues = decide_roles(state.get("blocks", []), state.get("sections", []), ctx.client, ctx.cfg)
 99      return {"roles": roles}, issues
100
101
102  def stage_recipes(ctx: RunContext, state: Dict[str, Any]) -> Tuple[Dict[str, Any], List[Dict[str, str]]]:
103      recipes, issues = build_recipes(state.get("blocks", []), state.get("roles", {}), ctx.symbols,
104                                      ctx.client, ctx.cfg, ctx.recipe_dir)
105      return {"recipes": recipes}, issues
106
107
108  def stage_assemble(ctx: RunContext, state: Dict[str, Any]) -> Tuple[Dict[str, Any], List[Dict[str, str]]]:
109      cases, issues = assemble_cases(state.get("blocks", []), state.get("roles", {}),
110                                     state.get("recipes", {}), ctx.cfg)
111      return {"cases": cases}, issues
112
113
114  def stage_notes(ctx: RunContext, state: Dict[str, Any]) -> Tuple[Dict[str, Any], List[Dict[str, str]]]:
115      notes, issues = extract_notes(state.get("blocks", []), state.get("sections", []), ctx.client, ctx.cfg)
116      return {"notes": notes}, issues
117
118
119  def _attach_symbols(ctx: RunContext, cases: List[Dict[str, Any]], issues: List[Dict[str, str]]) -> None:
120      """Give every item its symbol; a variable that is not in symbols.csv becomes unresolved."""
121      for case in cases:
122          for step in case.get("steps", []):
123              for bucket in ("set_input", "check_expected_output", "verify_only"):
124                  for item in step.get(bucket, []):
125                      var = item.get("variable")
126                      if var and not ctx.symbols.has_variable(var):
127                          issues.append(make_issue("error", "ground", "variable %r is not in symbols.csv" % var,
128                                                   item.get("source", "")))
129                          item["variable"] = None
130                      item["symbol"] = ctx.symbols.symbol_for(item.get("variable"))
131
132
133  def stage_ground(ctx: RunContext, state: Dict[str, Any]) -> Tuple[Dict[str, Any], List[Dict[str, str]]]:
134      issues: List[Dict[str, str]] = []
135      cases = copy.deepcopy(state.get("cases", []))
136      _attach_symbols(ctx, cases, issues)
137      for case in cases:
138          grounded = []
139          for name in case.get("verify_test_case_environment", []):
140              text = name if isinstance(name, str) else name.get("variable_text", "")
141              var = _ground_name(ctx, text, "environment variable to verify in test case %s" % case["id"],
142                                 issues, "case %s" % case["id"])
143              grounded.append({"variable_text": text, "variable": var, "symbol": ctx.symbols.symbol_for(var)})
144          case["verify_test_case_environment"] = grounded
145      notes = state.get("notes", {})
146      init = notes.get("init", {})
147      g_init: Dict[str, Any] = {"funcs": [], "test_variables": [], "verify_variables": []}
148      known = ctx.cfg.get("funcs") or []
149      for f in init.get("funcs", []):
150          if known and f not in known:
151              issues.append(make_issue("review", "ground", "init function %r is not in the configured function list" % f
      +, "global init"))
152              continue
153          g_init["funcs"].append(f)
154      for t in init.get("test_variables", []):
155          var = _ground_name(ctx, t["variable"], "variable checked during global initialization", issues, "global init")
156          g_init["test_variables"].append({"variable_text": t["variable"], "expected_value": t["expected_value"],
157                                           "variable": var, "symbol": ctx.symbols.symbol_for(var)})
158      for v in init.get("verify_variables", []):
159          var = _ground_name(ctx, v["variable"], "variable verified manually during global initialization", issues, "glo
      +bal init")
160          g_init["verify_variables"].append({"variable_text": v["variable"], "variable": var,
161                                             "symbol": ctx.symbols.symbol_for(var)})
162      spec = {"version": 1,
163              "meta": {"source": os.path.basename(ctx.input_path), "model": ctx.client.model_id,
164                       "generated": time.strftime("%Y-%m-%d %H:%M:%S"), "n_cases": len(cases)},
165              "global_init": g_init,
166              "constraints": [c.get("text", "") for c in notes.get("constraints", [])],
167              "test_cases": cases}
```

```
    write_json(os.path.join(ctx.out_dir, ctx.stem + ".spec.json"), spec)
    return {"spec": spec}, issues


def stage_validate(ctx: RunContext, state: Dict[str, Any]) -> Tuple[Dict[str, Any], List[Dict[str, str]]]:
    issues = validate_spec(state.get("spec", {}), state.get("sections", []), state.get("roles", {}),
                           state.get("recipes", {}), ctx.cfg)
    return {"validation": issues}, issues


def stage_render(ctx: RunContext, state: Dict[str, Any]) -> Tuple[Dict[str, Any], List[Dict[str, str]]]:
    lines, issues, stats = render_spec(state.get("spec", {}), ctx.cfg)
    path = ctx.path(".tpf")
    os.makedirs(ctx.out_dir, exist_ok=True)
    tmp = path + ".tmp"
    with open(tmp, "w", encoding="utf-8", newline="\n") as handle:
        handle.write("\n".join(lines) + "\n")
    os.replace(tmp, path)
    return {"outputs": {"tpf": path, "render_stats": stats}}, issues


STAGES: List[Tuple[str, Callable[..., Any]]] = [
    ("load", stage_load), ("sections", stage_sections), ("roles", stage_roles),
    ("recipes", stage_recipes), ("assemble", stage_assemble), ("notes", stage_notes),
    ("ground", stage_ground), ("validate", stage_validate), ("render", stage_render),
]
STAGE_NAMES = [n for n, _ in STAGES]


# ---------------------------------------------------------------------------
# fallbacks: what a stage returns when it crashed (must never raise itself)
# ---------------------------------------------------------------------------
def _disabled_client() -> LLMClient:
    return LLMClient(None, model_id="none")


def fallback_for(ctx: RunContext, name: str, state: Dict[str, Any]) -> Tuple[Dict[str, Any], List[Dict[str, str]]]:
    try:
        if name == "sections":
            sections = build_sections(state.get("blocks", []), int(ctx.cfg["sections"]["preview_chars"]))
            return {"sections": sections}, label_sections(sections, _disabled_client(), ctx.cfg)
        if name == "roles":
            roles, iss = decide_roles(state.get("blocks", []), state.get("sections", []), _disabled_client(), ctx.cfg)
            return {"roles": roles}, iss
        if name == "recipes":
            return {"recipes": {}}, []
        if name == "assemble":
            return {"cases": []}, []
        if name == "notes":
            return {"notes": {"init": {"funcs": [], "test_variables": [], "verify_variables": [], "source_sections": [
      +]},
                              "constraints": [], "raw": {"initialization": [], "constraints": []}}}, []
        if name == "ground":
            cases = copy.deepcopy(state.get("cases", []))
            iss: List[Dict[str, str]] = []
            _attach_symbols(ctx, cases, iss)
            for case in cases:
                case["verify_test_case_environment"] = [
                    {"variable_text": str(v), "variable": None, "symbol": None}
                    for v in case.get("verify_test_case_environment", [])]
            return {"spec": {"version": 1, "meta": {"source": os.path.basename(ctx.input_path)},
                             "global_init": {}, "constraints": [], "test_cases": cases}}, iss
        if name == "validate":
            return {"validation": []}, []
        if name == "render":
            path = ctx.path(".tpf")
            os.makedirs(ctx.out_dir, exist_ok=True)
            with open(path, "w", encoding="utf-8") as handle:
                handle.write("# RENDER FAILED - see review.md and work/errors.log\n")
            return {"outputs": {"tpf": path, "render_stats": {}}}, []
    except Exception:                                   # noqa: BLE001
        pass
    return {}, []


# ---------------------------------------------------------------------------
# node wrapper
# ---------------------------------------------------------------------------
def _ckpt_path(ctx: RunContext, idx: int, name: str) -> str:
    return os.path.join(ctx.work_dir, "%02d_%s.json" % (idx, name))


def make_node(ctx: RunContext, idx: int, name: str, fn: Callable[..., Any]) -> Callable[[Dict[str, Any]], Dict[str, An
  +y]]:
    def node(state: Dict[str, Any]) -> Dict[str, Any]:
        if state.get("fatal"):
            return {"stage_log": [{"stage": name, "status": "skipped", "seconds": 0.0}]}
        started = time.time()
        status = "ok"
```

```
        try:
            if ctx.resume_from is not None and idx < ctx.resume_from:
                path = _ckpt_path(ctx, idx, name)
                if not os.path.isfile(path):
                    raise FatalError("cannot resume from stage %d: checkpoint %s is missing; run the earlier stages fi
      +rst"
                                     % (ctx.resume_from, path))
                try:
                    with open(path, "r", encoding="utf-8") as handle:
                        saved = json.load(handle)
                    update, issues = saved["update"], saved.get("issues", [])
                except (OSError, ValueError, KeyError) as exc:
                    raise FatalError("checkpoint %s is unreadable (%s)" % (path, exc))
                status = "resumed"
            else:
                update, issues = fn(ctx, state)
                try:
                    write_json(_ckpt_path(ctx, idx, name), {"stage": name, "update": update, "issues": issues})
                except OSError:
                    issues = list(issues) + [make_issue("info", name, "checkpoint could not be written")]
        except FatalError as exc:
            return {"fatal": str(exc), "issues": [make_issue("error", name, str(exc))],
                    "stage_log": [{"stage": name, "status": "fatal", "seconds": round(time.time() - started, 3)}]}
        except Exception as exc:                        # noqa: BLE001 - a stage must never kill the run
            trace = traceback.format_exc()
            try:
                os.makedirs(ctx.work_dir, exist_ok=True)
                with open(os.path.join(ctx.work_dir, "errors.log"), "a", encoding="utf-8") as handle:
                    handle.write("=== stage %s ===\n%s\n" % (name, trace))
            except OSError:
                pass
            msg = "stage %r failed: %s: %s" % (name, type(exc).__name__, exc)
            if ctx.strict or name == "load":
                return {"fatal": msg, "issues": [make_issue("error", name, msg)],
                        "stage_log": [{"stage": name, "status": "fatal", "seconds": round(time.time() - started, 3)}]}
            update, extra = fallback_for(ctx, name, state)
            issues = [make_issue("error", name, msg, fix="see work/errors.log; a rule-based fallback was used")] + lis
      +t(extra)
            status = "failed"
            try:
                write_json(_ckpt_path(ctx, idx, name), {"stage": name, "update": update, "issues": issues})
            except OSError:
                pass
        out: Dict[str, Any] = dict(update)
        out["issues"] = list(issues)
        out["stage_log"] = [{"stage": name, "status": status, "seconds": round(time.time() - started, 3)}]
        if ctx.strict and status != "resumed" and any(i["severity"] == "error" for i in issues):
            out["fatal"] = "strict mode: stage %r reported errors" % name
        return out
    node.__name__ = "node_" + name
    return node
```

testgen/graph.py **(97 lines)**

```
"""LangGraph wiring.

    START -> load -> sections -> roles -> recipes -> assemble -> notes
          -> ground -> validate -> render -> END
with a conditional edge after every node: a fatal error or --stop-after jumps
straight to END.

Engines
    langgraph   the real langgraph.graph.StateGraph
    sequential  a 30-line runner with the same node functions and reducers, used
                automatically when langgraph is not installed (engine = "auto")
Both engines run the same node functions, so results are identical. If the
LangGraph engine itself raises (not a stage - stages never raise), "auto" mode
logs it and re-runs with the sequential engine.
"""
from __future__ import annotations

from typing import Any, Callable, Dict, List, Optional

from .issues import FatalError
from .stages import STAGES, RunContext, make_node
from .state import REDUCERS, RunState

END_NODE = "__end__"       # same value as langgraph.graph.END


def build_nodes(ctx: RunContext) -> List[Any]:
    return [(name, make_node(ctx, i, name, fn)) for i, (name, fn) in enumerate(STAGES)]


def make_router(ctx: RunContext, idx: int, next_name: str, end: Any) -> Callable[[Dict[str, Any]], Any]:
    def route(state: Dict[str, Any]) -> Any:
        if state.get("fatal"):
            return end
```

```
        if ctx.stop_after is not None and idx >= ctx.stop_after:
            return end
        return next_name
    return route


class SequentialApp:
    """Minimal stand-in for a compiled graph: invoke(state) -> final state."""

    def __init__(self, nodes: List[Any], routers: List[Callable[[Dict[str, Any]], Any]]) -> None:
        self.nodes = nodes
        self.routers = routers

    def invoke(self, state: Dict[str, Any], config: Optional[Dict[str, Any]] = None) -> Dict[str, Any]:
        state = dict(state)
        index = {name: i for i, (name, _) in enumerate(self.nodes)}
        current = self.nodes[0][0]
        for _ in range(len(self.nodes) + 2):           # hard stop against routing bugs
            if current == END_NODE:
                break
            i = index[current]
            update = self.nodes[i][1](state) or {}
            for key, value in update.items():
                if key in REDUCERS:
                    state[key] = REDUCERS[key](state.get(key, []), value)
                else:
                    state[key] = value
            current = self.routers[i](state)
        return state


def build_langgraph_app(ctx: RunContext) -> Any:
    from langgraph.graph import END, START, StateGraph      # raises ImportError if missing
    nodes = build_nodes(ctx)
    graph = StateGraph(RunState)
    for name, fn in nodes:
        graph.add_node(name, fn)
    graph.add_edge(START, nodes[0][0])
    for i, (name, _) in enumerate(nodes):
        nxt = nodes[i + 1][0] if i + 1 < len(nodes) else END
        route = make_router(ctx, i, nxt, END)
        graph.add_conditional_edges(name, route, {nxt: nxt, END: END})
    return graph.compile()


def build_sequential_app(ctx: RunContext) -> SequentialApp:
    nodes = build_nodes(ctx)
    routers = []
    for i in range(len(nodes)):
        nxt = nodes[i + 1][0] if i + 1 < len(nodes) else END_NODE
        routers.append(make_router(ctx, i, nxt, END_NODE))
    return SequentialApp(nodes, routers)


def build_app(ctx: RunContext, engine: str = "auto") -> Any:
    """Return (app, engine_used)."""
    if engine in ("auto", "langgraph"):
        try:
            return build_langgraph_app(ctx), "langgraph"
        except ImportError as exc:
            if engine == "langgraph":
                raise FatalError("engine 'langgraph' requested but langgraph is not installed (%s)" % exc)
    return build_sequential_app(ctx), "sequential"
```

`testgen/report.py` **(118 lines)**

```
"""Human-readable outputs: review.md, tables.md, run_report.json.

review.md is the file to read after every run: errors first, then review items,
then what the pipeline decided (section labels, table roles, recipes) and the
paragraph facts that were not turned into test lines.
"""
from __future__ import annotations

import json
import os
from typing import Any, Dict, List

from .issues import counts, dedupe, exit_code
from .stages import RunContext, write_json
from .table_decoder import to_markdown


def _issue_line(i: Dict[str, Any]) -> str:
    loc = (" `%s`" % i["location"]) if i.get("location") else ""
    fix = (" -> %s" % i["fix"]) if i.get("fix") else ""
    return "- [%s]%s %s%s" % (i["stage"], loc, i["message"], fix)

```

```
23
24  def build_review(ctx: RunContext, state: Dict[str, Any], issues: List[Dict[str, Any]], code: int,
25                   engine: str) -> str:
26      c = counts(issues)
27      out: List[str] = ["# Review: %s" % ctx.stem, ""]
28      status = {0: "clean", 1: "finished, items need review", 2: "FATAL - run did not finish"}[code]
29      out.append("Status: **%s** | errors %d | review %d | info %d | engine %s | model %s"
30                 % (status, c["error"], c["review"], c["info"], engine, ctx.client.model_id))
31      if state.get("fatal"):
32          out += ["", "**Fatal:** %s" % state["fatal"]]
33      for sev, title in (("error", "Errors"), ("review", "Please review")):
34          items = [i for i in issues if i["severity"] == sev]
35          if items:
36              out += ["", "## %s (%d)" % (title, len(items))] + [_issue_line(i) for i in items]
37      sections = state.get("sections", [])
38      if sections:
39          out += ["", "## Section labels", "", "| id | heading | label | source |", "|---|---|---|---|"]
40          for s in sections:
41              out.append("| %s | %s | %s | %s |" % (s["id"], s["heading"].replace("|", "/"), s["label"], s["label_source
      +"]))
42      roles = state.get("roles", {})
43      if roles:
44          out += ["", "## Table roles", "", "| table block | role | source | reason |", "|---|---|---|---|"]
45          for tid, r in roles.items():
46              out.append("| %s | %s | %s | %s |" % (tid, r["role"], r["source"], str(r["reason"]).replace("|", "/")))
47      recipes = state.get("recipes", {})
48      if recipes:
49          out += ["", "## Recipes (column meanings)"]
50          for tid, rec in recipes.items():
51              out += ["", "### %s (layout %s, %s, approved=%s)" % (tid, rec["layout_hash"], rec["role"], rec.get("approv
      +ed")),
52                      "", "| column | kind | variable | source |", "|---|---|---|---|"]
53              for key, e in rec["fields"].items():
54                  out.append("| %s | %s | %s | %s |" % (key.replace("|", "/"), e["kind"], e.get("variable") or "-", e["s
      +ource"]))
55      notes = state.get("notes", {})
56      spec = state.get("spec", {})
57      if notes:
58          init = notes.get("init", {})
59          out += ["", "## Facts from paragraphs (not turned into test lines unless emit_global_init is true)"]
60          out.append("- init functions: %s" % (", ".join(init.get("funcs", [])) or "none"))
61          for t in spec.get("global_init", {}).get("test_variables", []):
62              out.append("- init check: %s = %s (symbol %s)" % (t["variable_text"], t["expected_value"], t.get("symbol")
      + or "UNRESOLVED"))
63          for v in spec.get("global_init", {}).get("verify_variables", []):
64              out.append("- init verify: %s (symbol %s)" % (v["variable_text"], v.get("symbol") or "UNRESOLVED"))
65          for ctext in spec.get("constraints", []):
66              out.append("- constraint: %s" % ctext)
67          for label, lines in notes.get("raw", {}).items():
68              if lines:
69                  out += ["", "Raw %s text:" % label] + ["> " + l for l in lines]
70      info = [i for i in issues if i["severity"] == "info"]
71      if info:
72          out += ["", "## Info (%d)" % len(info)] + [_issue_line(i) for i in info]
73      stats = ctx.client.stats
74      out += ["", "## LLM usage", "- calls %(calls)d, attempts %(attempts)d, cache hits %(cache_hits)d, retries %(retrie
      +s)d, "
75              "failures %(failures)d, truncated %(truncated)d, timeouts %(timeouts)d, seconds %(seconds).1f" % stats]
76      return "\n".join(out) + "\n"
77
78
79  def build_tables_md(state: Dict[str, Any]) -> str:
80      roles = state.get("roles", {})
81      out = ["# Decoded tables", "", "Each test row is shown as its own small table. Check the decoder here first."]
82      for b in state.get("blocks", []):
83          if b["kind"] != "table":
84              continue
85          t = b["table"]
86          role = roles.get(b["id"], {}).get("role", "?")
87          out += ["", "## Table %d (%s) role: %s" % (b["table_index"] + 1, b["id"], role)]
88          if t is None:
89              out.append("_could not be decoded; text: %s_" % b["text"][:200])
90              continue
91          out.append("layout %s, %d header row(s), %d column(s), %d data row(s), id column: %s"
92                     % (t["layout_hash"], t["n_header_rows"], t["n_cols"], t["n_rows"], t["id_column"]))
93          out += ["", to_markdown(t, "per_case")]
94      return "\n".join(out) + "\n"
95
96
97  def write_reports(ctx: RunContext, state: Dict[str, Any], engine: str) -> int:
98      """Write review.md, tables.md and run_report.json. Returns the exit code."""
99      issues = dedupe(state.get("issues", []))
100     code = exit_code(issues, bool(state.get("fatal")))
101     os.makedirs(ctx.out_dir, exist_ok=True)
102     for suffix, builder in ((".review.md", lambda: build_review(ctx, state, issues, code, engine)),
103                             (".tables.md", lambda: build_tables_md(state))):
104         try:
105             with open(ctx.path(suffix), "w", encoding="utf-8") as handle:
106                 handle.write(builder())
```

```
        except Exception as exc:                        # noqa: BLE001 - reports are best effort
            issues.append({"severity": "info", "stage": "report", "location": suffix,
                           "message": "could not write report: %s" % exc, "fix": ""})
    try:
        write_json(ctx.path(".run_report.json"), {
            "input": ctx.input_path, "engine": engine, "exit_code": code, "fatal": state.get("fatal"),
            "counts": counts(issues), "stages": state.get("stage_log", []),
            "llm": dict(ctx.client.stats, model=ctx.client.model_id),
            "outputs": state.get("outputs", {})})
    except OSError:
        pass
    return code
```

testgen/runner.py **(43 lines)**

```
"""Run the whole pipeline once. This is the function the CLI and the tests call."""
from __future__ import annotations

import os
from typing import Any, Dict, Optional, Tuple

from .graph import build_app, build_sequential_app
from .grounding import SymbolTable
from .issues import FatalError, make_issue
from .llm_client import LLMClient
from .report import write_reports
from .stages import RunContext


def run_pipeline(input_path: str, out_dir: str, cfg: Dict[str, Any], symbols: SymbolTable,
                 client: LLMClient, recipe_dir: Optional[str] = None, strict: bool = False,
                 resume_from: Optional[int] = None, stop_after: Optional[int] = None,
                 engine: Optional[str] = None, config_issues: Optional[list] = None
                 ) -> Tuple[Dict[str, Any], int, str]:
    """Return (final_state, exit_code, engine_used). Never raises for document problems."""
    ctx = RunContext(cfg, symbols, client, input_path, out_dir, recipe_dir=recipe_dir, strict=strict,
                     resume_from=resume_from, stop_after=stop_after)
    os.makedirs(ctx.work_dir, exist_ok=True)
    initial: Dict[str, Any] = {"input_path": input_path, "issues": list(config_issues or []),
                               "stage_log": [], "fatal": None}
    wanted = engine or cfg.get("engine", "auto")
    try:
        app, used = build_app(ctx, wanted)
    except FatalError as exc:
        state = dict(initial, fatal=str(exc))
        state["issues"] = state["issues"] + [make_issue("error", "engine", str(exc))]
        return state, write_reports(ctx, state, "none"), "none"
    try:
        state = app.invoke(initial, config={"recursion_limit": 60}) if used == "langgraph" else app.invoke(initial)
    except Exception as exc:                            # noqa: BLE001 - the graph engine itself failed
        if wanted == "langgraph":
            raise
        note = make_issue("error", "engine", "%s engine failed (%s: %s); re-ran with the sequential engine"
                          % (used, type(exc).__name__, exc))
        used = "sequential"
        state = build_sequential_app(ctx).invoke(dict(initial, issues=list(initial["issues"]) + [note]))
    code = write_reports(ctx, state, used)
    return state, code, used
```

testgen/eval.py **(33 lines)**

```
"""Compare a generated tpf with a gold tpf (line level, order-insensitive).

Comments (#...) and blank lines are ignored, whitespace is normalised. Gives
precision / recall / F1 plus the missing and extra lines, so you can see what to
fix in recipes or config.
"""
from __future__ import annotations

import re
from collections import Counter
from typing import Any, Dict, List


def _norm_lines(text: str) -> List[str]:
    out = []
    for raw in text.splitlines():
        line = raw.strip()
        if not line or line.startswith("#"):
            continue
        out.append(re.sub(r"\s+", " ", line))
    return out


def compare(generated: str, gold: str) -> Dict[str, Any]:
    g, t = Counter(_norm_lines(generated)), Counter(_norm_lines(gold))
    matched = sum((g & t).values())
```

```
    n_gen, n_gold = sum(g.values()), sum(t.values())
    precision = matched / n_gen if n_gen else 0.0
    recall = matched / n_gold if n_gold else 0.0
    f1 = 2 * precision * recall / (precision + recall) if (precision + recall) else 0.0
    return {"generated_lines": n_gen, "gold_lines": n_gold, "matched": matched,
            "precision": round(precision, 4), "recall": round(recall, 4), "f1": round(f1, 4),
            "missing": sorted((t - g).elements()), "extra": sorted((g - t).elements())}
```

testgen/testing.py **(109 lines)**

```
"""Fake model for tests and dry runs (no LLM needed).

FakeModel answers every prompt type the pipeline sends, by parsing the prompt,
like a competent small model would. `fail` injects failures per task:
    fail={"table_role": ("malformed", None)}   # always
    fail={"field_map": ("exception", 1)}       # only the first call
modes: malformed | bad_enum | truncated | exception | timeout | empty | fenced
"""
from __future__ import annotations

import json
import re
import time
from typing import Any, Callable, Dict, List, Optional, Tuple

from .llm_client import LLMTransportError, Reply


def _task(messages: List[Tuple[str, str]]) -> str:
    m = re.match(r"\s*TASK:\s*(\w+)", messages[-1][1])
    return m.group(1) if m else "unknown"


class FakeModel:
    def __init__(self, fail: Optional[Dict[str, Tuple[str, Optional[int]]]] = None,
                 overrides: Optional[Dict[str, Callable[[str], Any]]] = None, sleep_for: float = 0.3) -> None:
        self.fail = dict(fail or {})
        self.overrides = overrides or {}
        self.calls: List[str] = []
        self.sleep_for = sleep_for
        self._counts: Dict[str, int] = {}

    # -- Transport interface ------------------------------------------------
    def __call__(self, messages: List[Tuple[str, str]], schema: Optional[Dict[str, Any]] = None) -> Reply:
        task = _task(messages)
        self.calls.append(task)
        n = self._counts[task] = self._counts.get(task, 0) + 1
        prompt = messages[-1][1]
        mode = self.fail.get(task)
        if mode and (mode[1] is None or n <= mode[1]):
            return self._broken(mode[0], task, prompt)
        if task in self.overrides:
            res = self.overrides[task](prompt)
            return res if isinstance(res, Reply) else Reply(res if isinstance(res, str) else json.dumps(res))
        return Reply(self._answer(task, prompt))

    def _broken(self, mode: str, task: str, prompt: str) -> Reply:
        if mode == "exception":
            raise LLMTransportError("connection refused (fake)")
        if mode == "timeout":
            time.sleep(self.sleep_for)
            return Reply("{}")
        if mode == "malformed":
            return Reply("Sure! Here is my answer: it is probably the second one.")
        if mode == "empty":
            return Reply("")
        if mode == "truncated":
            return Reply('{"labels": [{"id": "S0', truncated=True)
        if mode == "bad_enum":
            return Reply(json.dumps({"labels": [{"id": "S000", "label": "banana"}], "role": "banana",
                                     "variable": "banana", "kind": "banana"}))
        if mode == "fenced":
            return Reply("```json\n" + self._answer(task, prompt) + "\n```")
        raise ValueError("unknown fail mode %r" % mode)

    # -- sensible answers ---------------------------------------------------
    def _answer(self, task: str, prompt: str) -> str:
        if task == "section_labels":
            labels = []
            for m in re.finditer(r"^\[(S\d+)\].*?hint: (\w+)", prompt, re.M):
                hint = m.group(2)
                labels.append({"id": m.group(1), "label": hint if hint not in ("none", "") else "other"})
            return json.dumps({"labels": labels})
        if task == "table_role":
            hint = (re.search(r"Rule-based hint: (\w+)", prompt) or [None, "ignore"])[1]
            return json.dumps({"reason": "fake", "role": hint if hint in ("input", "expected", "mixed", "ignore") else
      + "ignore"})
        if task == "field_map":
```

```
            role = re.search(r"Table role: (\w+)", prompt).group(1)
            block = re.search(r"Candidate variables:\n((?:- .*\n?)+)", prompt).group(1)
            names = [l[2:].strip() for l in block.splitlines() if l.startswith("- ") and "no candidates" not in l]
            if not names:
                return json.dumps({"reason": "none", "variable": "NONE", "kind": "ignore"})
            return json.dumps({"reason": "first candidate", "variable": names[0],
                               "kind": {"input": "set_input", "expected": "check"}.get(role, "check")})
        if task == "choose_variable":
            block = re.search(r"Candidates:\n((?:- .*\n?)+)", prompt).group(1)
            first = block.splitlines()[0][2:].split(" (")[0].strip()
            return json.dumps({"variable": first})
        if task == "notes_normalize":
            text = prompt.split("TEXT:\n", 1)[1]
            funcs = [w.upper() for w in ("boot", "load") if re.search(r"\b%s\b" % w, text, re.I)]
            checks = ["%s = %s" % (a, b) for a, b in re.findall(r"(\w+) is (\d+)", text)]
            verify = re.findall(r"the (\w+) value manually", text)
            cons = [s.strip() for s in text.split(".") if "must" in s]
            return ("FUNCTIONS: %s\nCHECKS: %s\nVERIFY: %s\nCONSTRAINTS: %s"
                    % (", ".join(funcs) or "NONE", "; ".join(checks) or "NONE", ", ".join(verify) or "NONE",
                       "; ".join(cons) or "NONE"))
        if task == "notes_extract":
            text = prompt.split("ORIGINAL TEXT:\n", 1)[1].split("\n\nNOTES:", 1)[0]
            funcs = [w.upper() for w in ("boot", "load") if re.search(r"\b%s\b" % w, text, re.I)]
            tests = [{"evidence": "%s is %s" % (a, b), "variable": a, "expected_value": b}
                     for a, b in re.findall(r"(\w+) is (\d+)", text)]
            verify = [{"evidence": "the %s value manually" % v, "variable": v}
                      for v in re.findall(r"the (\w+) value manually", text)]
            cons = [{"evidence": s.strip(), "text": s.strip()} for s in text.split(".") if "must" in s]
            return json.dumps({"funcs": funcs, "test_variables": tests, "verify_variables": verify, "constraints": con
        +s})
        if task == "ping":
            return json.dumps({"ok": True})
        raise ValueError("FakeModel has no answer for task %r" % task)
```

testgen/cli.py **(206 lines)**

```
"""Command line interface.

    python -m testgen run SPEC.docx --symbols symbols.csv [--config config.json] [--out DIR]
                          [--backend ollama|lmstudio|none] [--model NAME] [--base-url URL]
                          [--strict] [--resume-from N] [--stop-after N] [--engine auto|langgraph|sequential]
    python -m testgen tables SPEC.docx [--header-rows N] [--transpose] [--style per_case|flat]
    python -m testgen render SPEC.spec.json [--config config.json] [--out FILE.tpf]
    python -m testgen eval GENERATED.tpf GOLD.tpf
    python -m testgen check --symbols symbols.csv [--config config.json] [--backend ...]

Exit codes: 0 clean | 1 finished with items to review | 2 fatal.
"""
from __future__ import annotations

import argparse
import json
import os
import sys
from typing import Any, Dict, List, Optional

from .config import load_config
from .eval import compare
from .grounding import load_symbols
from .issues import FatalError, counts
from .llm_client import LLMError, build_client
from .render import render_spec
from .runner import run_pipeline
from .stages import STAGE_NAMES


def _overrides(args: argparse.Namespace) -> Dict[str, Any]:
    llm: Dict[str, Any] = {}
    if getattr(args, "backend", None):
        llm["backend"] = args.backend
    if getattr(args, "model", None):
        llm["model"] = args.model
    if getattr(args, "base_url", None):
        llm["base_url"] = args.base_url
    out: Dict[str, Any] = {"llm": llm} if llm else {}
    if getattr(args, "strict", False):
        out["strict"] = True
    return out


def _stage_index(value: Optional[str]) -> Optional[int]:
    if value is None:
        return None
    if value.isdigit() and int(value) < len(STAGE_NAMES):
        return int(value)
    if value in STAGE_NAMES:
        return STAGE_NAMES.index(value)
    raise FatalError("stage must be 0-%d or one of %s" % (len(STAGE_NAMES) - 1, ", ".join(STAGE_NAMES)))
```

```
 53
 54
 55 def cmd_run(args: argparse.Namespace) -> int:
 56     cfg, cfg_issues = load_config(args.config, _overrides(args))
 57     symbols, sym_issues = load_symbols(args.symbols)
 58     out_dir = args.out or os.path.splitext(args.spec)[0] + "_out"
 59     base = os.path.dirname(os.path.abspath(args.config)) if args.config else os.getcwd()
 60     recipe_dir = args.recipes or cfg["recipe_dir"]
 61     if not os.path.isabs(recipe_dir):
 62         recipe_dir = os.path.join(base, recipe_dir)
 63     client = build_client(cfg, os.path.join(out_dir, "work"))
 64     state, code, engine = run_pipeline(
 65         args.spec, out_dir, cfg, symbols, client, recipe_dir=recipe_dir, strict=args.strict,
 66         resume_from=_stage_index(args.resume_from), stop_after=_stage_index(args.stop_after),
 67         engine=args.engine, config_issues=cfg_issues + sym_issues)
 68     c = counts(state.get("issues", []))
 69     print("engine: %s | model: %s" % (engine, client.model_id))
 70     if state.get("fatal"):
 71         print("FATAL: %s" % state["fatal"], file=sys.stderr)
 72     else:
 73         print("test file: %s" % state.get("outputs", {}).get("tpf", "(not rendered)"))
 74     print("errors %d | review %d | info %d  -> read %s" % (
 75         c["error"], c["review"], c["info"], os.path.join(out_dir, os.path.splitext(os.path.basename(args.spec))[0] + "
       +.review.md")))
 76     return code
 77
 78
 79 def cmd_tables(args: argparse.Namespace) -> int:
 80     from .docx_loader import load_blocks
 81     from .table_decoder import to_markdown
 82     cfg, _ = load_config(args.config)
 83     if args.header_rows:
 84         cfg["table_options"]["header_rows"] = args.header_rows
 85     if args.transpose:
 86         cfg["table_options"]["transpose"] = True
 87     blocks, issues = load_blocks(args.spec, cfg)
 88     for b in blocks:
 89         if b["kind"] == "table":
 90             print("\n## Table %d" % (b["table_index"] + 1))
 91             if b["table"] is None:
 92                 print("(could not be decoded)")
 93                 continue
 94             t = b["table"]
 95             print("%d header rows, %d columns, %d data rows, id column: %s" % (t["n_header_rows"], t["n_cols"], t["n_r
       +ows"], t["id_column"]))
 96             print(to_markdown(t, args.style))
 97     for i in issues:
 98         print("[%s] %s" % (i["severity"], i["message"]), file=sys.stderr)
 99     return 0
100
101
102 def cmd_render(args: argparse.Namespace) -> int:
103     cfg, _ = load_config(args.config)
104     try:
105         with open(args.spec_json, "r", encoding="utf-8") as handle:
106             spec = json.load(handle)
107     except (OSError, ValueError) as exc:
108         raise FatalError("cannot read %s: %s" % (args.spec_json, exc))
109     lines, issues, stats = render_spec(spec, cfg)
110     out = args.out or os.path.splitext(args.spec_json)[0].replace(".spec", "") + ".tpf"
111     with open(out, "w", encoding="utf-8", newline="\n") as handle:
112         handle.write("\n".join(lines) + "\n")
113     print("wrote %s (%d lines, %d unresolved, %d todo)" % (out, stats["lines"], stats["unresolved"], stats["todo"]))
114     return 1 if issues else 0
115
116
117 def cmd_eval(args: argparse.Namespace) -> int:
118     try:
119         with open(args.generated, encoding="utf-8") as h1, open(args.gold, encoding="utf-8") as h2:
120             gen, gold = h1.read(), h2.read()
121     except OSError as exc:
122         raise FatalError("cannot read file: %s" % exc)
123     res = compare(gen, gold)
124     print("precision %.3f recall %.3f f1 %.3f (matched %d of %d generated, %d gold)" % (
125         res["precision"], res["recall"], res["f1"], res["matched"], res["generated_lines"], res["gold_lines"]))
126     for label in ("missing", "extra"):
127         for line in res[label][:30]:
128             print("%s: %s" % (label, line))
129     return 0
130
131
132 def cmd_check(args: argparse.Namespace) -> int:
133     cfg, issues = load_config(args.config, _overrides(args))
134     symbols, sym_issues = load_symbols(args.symbols)
135     print("config ok; %d symbols loaded" % len(symbols))
136     for i in issues + sym_issues:
137         print("[%s] %s" % (i["severity"], i["message"]))
138     client = build_client(cfg, None)
139     if not client.enabled:
```

```
        print("LLM backend: none (rules only)")
        return 0
    try:
        ans = client.ask_json("ping", "Answer only with JSON.", 'TASK: ping\nReturn {"ok": true}.',
                              {"type": "object", "properties": {"ok": {"type": "boolean"}},
                               "required": ["ok"], "additionalProperties": False}, retries=0)
        print("LLM backend %s answered: %s" % (client.model_id, ans))
        return 0
    except LLMError as exc:
        print("LLM backend %s FAILED: %s" % (client.model_id, exc), file=sys.stderr)
        return 2


def build_parser() -> argparse.ArgumentParser:
    p = argparse.ArgumentParser(prog="testgen", description="Word test description -> tpf test file")
    sub = p.add_subparsers(dest="command", required=True)

    def llm_args(sp: argparse.ArgumentParser) -> None:
        sp.add_argument("--config")
        sp.add_argument("--backend", choices=["ollama", "lmstudio", "none"])
        sp.add_argument("--model")
        sp.add_argument("--base-url", dest="base_url")

    r = sub.add_parser("run", help="run the full pipeline")
    r.add_argument("spec")
    r.add_argument("--symbols", required=True)
    r.add_argument("--out")
    r.add_argument("--recipes")
    r.add_argument("--strict", action="store_true")
    r.add_argument("--resume-from", dest="resume_from")
    r.add_argument("--stop-after", dest="stop_after")
    r.add_argument("--engine", choices=["auto", "langgraph", "sequential"])
    llm_args(r)
    r.set_defaults(func=cmd_run)
    t = sub.add_parser("tables", help="show how tables are decoded (no LLM)")
    t.add_argument("spec")
    t.add_argument("--config")
    t.add_argument("--header-rows", type=int, dest="header_rows")
    t.add_argument("--transpose", action="store_true")
    t.add_argument("--style", choices=["per_case", "flat"], default="per_case")
    t.set_defaults(func=cmd_tables)
    rd = sub.add_parser("render", help="re-render a hand-edited spec.json")
    rd.add_argument("spec_json")
    rd.add_argument("--config")
    rd.add_argument("--out")
    rd.set_defaults(func=cmd_render)
    e = sub.add_parser("eval", help="compare generated and gold tpf")
    e.add_argument("generated")
    e.add_argument("gold")
    e.set_defaults(func=cmd_eval)
    c = sub.add_parser("check", help="check config, symbols and the LLM connection")
    c.add_argument("--symbols", required=True)
    llm_args(c)
    c.set_defaults(func=cmd_check)
    return p


def main(argv: Optional[List[str]] = None) -> int:
    args = build_parser().parse_args(argv)
    try:
        return args.func(args)
    except FatalError as exc:
        print("FATAL: %s" % exc, file=sys.stderr)
        return 2
    except KeyboardInterrupt:
        print("interrupted", file=sys.stderr)
        return 130
```

`testgen/__main__.py` **(5 lines)**

```
import sys

from .cli import main

sys.exit(main())
```

Appendix: test files

Included in the zip under tests/: test_units.py, test_decoder.py, test_llm_client.py, test_pipeline.py, synth_docs.py (builds merged-header Word documents), fake_langgraph.py (LangGraph stand-in). Run: `python -m unittest discover -s tests -t .`